

A Systematic Mapping Study of AI Coding Assistants and Agents in Software Engineering: Supply-Side Capabilities versus Demand-Side Empirical Evidence

Empirical Research Project (ERP) 2 by

THIA Boon Sing

School of Computing and Information Systems, Singapore Management University

Supervised by: Prof. TA Nguyen Binh Duong and Prof. ZHU Feida

ABSTRACT

AI systems for software engineering (AI-SE) have evolved rapidly since 2021, from inline autocompletion to multi-step agentic execution. Recent surveys [48, 71, 73] map the supply side of this transition, covering agent capabilities, architectures, and evaluation methods. A parallel conceptual contribution by Treude and Gerosa [66] catalogues eleven developer–AI interaction types along a UI-trigger-mechanism axis, with empirical validation explicitly positioned as future work. What remains missing is a corpus-level synthesis of how software engineers, students, and citizen developers use AI-SE systems in practice. This systematic mapping study addresses that demand-side gap through three research questions. Research Question (RQ)1 asks what the landscape of empirical AI-SE research looks like. RQ2 asks what human-AI interaction modes the literature reports. RQ3 asks where gaps lie between the capabilities emphasized in supply-side surveys and the SDLC activities for which empirical evidence has accumulated. Under a pre-registered protocol, the study screens Scopus and the ACM Digital Library and completes backward snowballing from five seed surveys [1, 42, 48, 71, 73], producing a corpus of primary empirical papers published between 2022 and 2026. Each paper is classified along five facets: Contribution Type, Research Methodology, Population and Context, SDLC Activity, and Agent/Tool Profile. From this corpus the study delivers three linked contributions: a five-facet landscape map for RQ1; a five-mode Human-AI Interaction Mode Taxonomy for RQ2, i.e. Inline Completion, Conversational Prompting, Visual/Declarative Composition, Review and Validation, and Delegated Task Execution, ordered along a delegation-depth axis and supported by mode-layer coding saturation as a dependability check; and a capability–evidence cross-tabulation for RQ3 that converts every under-evidenced or empty intersection into a pre-registered study call within a research agenda for downstream qualitative investigation. The headline finding is that supply-side literature describing what AI systems can do has outpaced demand-side literature documenting how practitioners use them - a pattern we term supply-saturation versus demand-under-evidence. The mismatch is sharpest at early-lifecycle activities (requirements, design), at operational activities (CI/CD, project management), and across the Low-code and No-code paradigms; the research agenda names these intersections as priorities for tool builders, educators, and researchers. To support transparency and computational reproducibility, the complete research repository for this study is available at <https://github.com/alphathia/erp2-submit.git>.

CCS CONCEPTS • Software engineering • Software development techniques • Human-centered computing • Artificial intelligence Agents

Additional Keywords and Phrases: software engineering agents, low-code development agents, no-code development agents, systematic mapping study, AI-assisted coding, human-AI interaction, usage patterns, agentic software engineering

1 INTRODUCTION

1.1 Motivations

Since 2021, AI in software engineering has rapidly progressed from simple, one-line autocompletion to multi-step autonomous agentic execution. While GitHub Copilot popularized inline completion [21] as a seamless workflow integration, the landscape has expanded to include IDE-integrated chat assistants and agentic frameworks. Current commercial agents, such as Cursor, Windsurf, and Claude Code, now move beyond suggestions to propose, plan, execute, and iterate on code across the entire software development lifecycle (SDLC).

Three recent surveys examine the supply side of this technological transition. Wang et al. [71] classify agentic programming systems based on their behavioral dimensions. A second survey by Wang et al. [73] characterizes internal agent architectures through a perception–memory–action framework derived from a review of primary studies. Similarly, Otoum & Elkhaili [48] provide a methods-oriented systematic review of current agentic software engineering techniques. While these surveys collectively describe the capabilities and construction of AI systems, they do not synthesize corpus-level evidence of how professional software engineers, students, or citizen developers utilize these systems in practice.

As a result, a demand-side gap remains in the current literature. Previous technological shifts in software engineering, including Agile, DevOps, and the low-code/no-code platforms recently surveyed by Ajimati et al. [1], consistently demonstrate that technological capability and actual adoption often diverge. A supply-side only perspective cannot address the critical questions of practitioners, educators, and tool-builders regarding the distribution of demand: specifically, where empirical evidence of usage has accumulated, which interaction patterns are being reported, and which intersections of capability and activity remain under-evidenced.

1.2 Research Questions

To structure a demand-side synthesis of the 2022–2026 empirical AI-SE literature, this study poses three research questions:

- **RQ1. landscape:** What is the landscape of empirical research reporting how software engineers, students, and citizen developers use AI coding assistants, SE agents, and AI-augmented low-code and no-code platforms?
- **RQ2. Interaction modes.** What human-AI interaction modes are reported in the empirical literature, and how do they vary across SDLC activities, agent / tool profiles, and population and context?
- **RQ3. Capability-evidence gaps:** Where do gaps exist between the capabilities emphasized in recent supply-side surveys and the SDLC activities for which empirical evidence has accumulated?

1.3 Objectives

The study pursues four research objectives declared in its pre-registered protocol: (i) an evidence landscape map of empirical AI-SE literature along a five-facet classification; (ii) a human–AI interaction mode taxonomy along a delegation-depth axis; (iii) a capability–evidence gap matrix that cross-tabulates supply-side capabilities against empirical SDLC evidence; and finally, an evidence-based research agenda for downstream qualitative investigation. Mapping those objectives to the questions above, the paper contributes:

1. **A 2022–2026 demand-side landscape (RQ1)** that provides a systematic mapping of 640 primary empirical papers published between 2022 and 2026, each classified along: Facet 1 (F1) - Contribution type, Facet 2

(F2) - research methodology, Facet 3 (F3) - population and context, Facet 4 (F4) - SDLC activity, and Facet 5 (F5) - agent / tool profile. The F1 distribution is dominated by Evaluation Research (71.7%) and Validation Research (22.3%); the F5 paradigm distribution is dominated by Pro-code tools (84%), with Low-code and No-code sub-corpora thin (5% and 11% respectively).

2. **A five-mode Human–AI Interaction Mode Taxonomy** (RQ2) that orders along a delegation-depth axis from assistive to autonomous engagement: It identifies five distinct interaction modes: 1) Inline Completion, 2) Conversational Prompting, 3) Visual/Declarative Composition, 4) Review & Validation, and 5) Delegated Task Execution. By identifying empirically recurring modes of human–AI interaction that can be read against SDLC activities, tool profiles, and populations, provides a common vocabulary for downstream demand-side study design. The taxonomy was inductively derived from the canonical codes extracted from a corpus of 290 papers; theoretical saturation was achieved at the mode layer within the final 10% of the sample. To facilitate application, the taxonomy includes operational definitions, distinguishing criteria, and paraphrased exemplars
3. **A capability–SDLC gap matrix** (RQ3) that harmonizes supply-side capabilities from the three reference surveys into 19 unified IDs and cross-tabulating them against nine SDLC activities. By quantifying, for every capability that supply-side surveys emphasize whether and where empirical demand-side evidence has accumulated, making the capability-versus-evidence asymmetrically visible at a granular level rather than at a survey level.

To address the practical implications of RQ3, the study proposes a pre-registered research agenda of seven Wieringa-typed [79] Calls for downstream studies. The agenda has two parts. Six corpus-internal Calls target the 42 flagged supply-versus-demand gaps: Calls 1–5 (Evaluation or Validation Research) cover the 13 highest-priority gaps at priority ≥ 1.00 , and Call 6 (integrative Evaluation Research) bundles a cluster of 10 moderate-priority gaps at priority = 0.50. A seventh Call extends the agenda beyond the corpus-internal gap matrix as a downstream taxonomy construction and validation study addressing the Pro-code-dominance transferability boundary. Every priority ≥ 0.50 flagged gap maps to at least one Call, so the agenda names which empirical studies to run next, for which populations, and how the taxonomy should be extended beyond the Pro-code sub-corpus. Specifically, the seven Calls are:

- Call 1: AI code completion at early-lifecycle activities;
- Call 2: AI code translation beyond the coding boundary;
- Call 3: Agentic self-reflection outside Coding;
- Call 4: Multi-agent orchestration at under-studied SDLC intersections;
- Call 5: AI vulnerability detection beyond Coding;
- Call 6: Knowledge-management capabilities at non-native SDLC activities; and
- Call 7: A taxonomy of AI-agent tool architectures beyond the Pro-code-dominant corpus.

Each Call carries a research-question template and a population-fit note (Appendix C.5). Organizations, tool builders, educators, and researchers can adopt the agenda as a pre-registered prioritization rather than editorial commentary.

1.4 Paper Organization

Section 2 positions the study against the reference surveys and prior taxonomies of developer–AI interaction. Section 3 describes the study design: search, screening, classification, extraction, open coding, and the three analysis tracks. Section 4 reports findings by RQ. Section 5 discusses practical implications and presents the research agenda. Section 6 audits trustworthiness along the four qualitative-research criteria. Section 7 concludes. An appendix packages the codebook, per-mode operational definitions and exemplars, per-gap rationale, the comparison matrix, and the eight transferability boundaries.

2 BACKGROUND AND RELATED WORK

2.1 The supply-side reference surveys

Three recent surveys establish the foundation for this research. Wang et al. [71] review AI agentic programming, classifying systems across behavioral dimensions like tool augmentation and adaptability. A second survey by Wang et al. [73] examines software engineering agents through a perception-memory-action framework, while Otoum & Elkhaili [48] provide a methods-oriented review structured around lifecycle integration and evaluation. While all three acknowledge adoption as a critical open question and identify core capabilities, such as code completion, program repair, and multi-agent collaboration, none establishes a demand-side taxonomy or cross-tabulates these capabilities against empirical usage. This study addresses these limitations by synthesizing their disparate capability vocabularies into 19 unified identifiers (§3.5, Table 4) and cross-tabulating them against nine SDLC activities (§4.3).

2.2 Prior developer-AI interaction taxonomies

Two pieces of prior work directly address the developer-AI interaction-pattern question posed by RQ2. Treude & Gerosa [66] is a Solution Proposal paper that presented a conceptual taxonomy of author-identified interaction types rather than primary-source evidence of human interaction with AI-SE tools; Kumar et al. [37] is a qualitative observational study at a single company, Microsoft, whose empirical-usage character is in scope but whose single-site sampling fell below the corpus-inclusion rule. Both are therefore external methodology references rather than corpus papers; this is a standard positioning for mapping-study prior-work sections.

Treude & Gerosa [66] propose a conceptual taxonomy of eleven developer-AI interaction types at the user-interface mechanism level (auto-complete suggestions, command-driven actions, conversational assistance, contextual recommendations, selection-based enhancements, explicit UI actions, comment-guided prompts, event-based triggers, shortcut-activated commands, file-aware suggestions and automated API responses). The taxonomy is explicitly pre-empirical, not derived from a literature corpus and not cross-referenced against a supply-side capability inventory. The taxonomy is also accompanied by a research agenda of eight themes: (a) effectiveness of interaction types, (b) developer trust and adoption, (c) context-aware AI interactions, (d) optimizing for developer control and customization, (e) reducing cognitive load, (f) ethics and bias, (g) privacy and data protection, and (h) hallucination and damage control.

Of those eight, four are operationalized within the present agenda: theme (a) by Calls 1–6 at the capability $\times$ SDLC-activity level, theme (b) by Call 5’s security-alert triage study, theme (d) by the delegation-depth axis of §4.2 that quantifies developer-versus-agent control, and theme (h) by Call 3’s agentic-self-reflection study. The remaining four, i.e. (c) context-aware AI interaction, (e) cognitive load, (f) ethics and bias, (g) privacy, are deliberately scoped out of the demand-side mapping: (c) is a tool-design research direction that belongs to the design-side literature, and (e), (f), and (g) are cross-cutting human-factors covariates that the present study pre-registers in §5.3 as universal data-collection requirements rather than as standalone Calls.

Kumar et al. [36] contribute a qualitative observational study of 19 professional developers using Cursor Agent to resolve 33 open issues in repositories they had previously contributed to. The study is one of the first in-the-wild empirical accounts of developer-agent collaboration in an integrated development environment. The study organizes its findings around three questions: Kumar-RQ1 (collaboration patterns) - how developers collaborate with a SWE agent, Kumar-RQ2 (barriers) - what barriers developers face in that collaboration, and Kumar-RQ3 (success factors) - what factors predict success.

Their empirical findings inform the present research agenda in two concrete ways. First, Kumar-RQ1 identifies two distinct delegation strategies that developers adopt when working with a SWE agent: 1) a one-shot strategy in which the developer hands the task to the agent as a single prompt, and 2) an incremental-resolution strategy in which the developer and agent iterate through sub-tasks; and Kumar-RQ3 reports their differential success rates. Of the 29

eligible issues (the 33 attempted minus 4 abandoned for reasons external to the agent), 83% of issues attempted under the incremental-resolution strategy reached satisfactory resolution, compared with 38% of issues attempted under the one-shot strategy; and 64% of the 22 issues where developers supplied expert insights reached satisfactory resolution, compared with 29% of the 7 issues where no expert insights were supplied. These observations give Mode 5 Delegated Task Execution (§4.2) an empirical interior structure at prompt-turn granularity that the present SMS's corpus-level synthesis could not produce by itself, and Mode 5 therefore carries Kumar et al.'s strategy distinction forward as a sub-mode refinement. Second, the Kumar-RQ2 barrier findings: lack of tacit knowledge, unsolicited agent actions, sycophancy, and overconfidence, are adopted as pre-registered measurement targets in Calls 3 and 4 of the present research agenda (§5.3), so that any downstream study executing those Calls carries forward Kumar et al.'s documented failure modes as explicit falsifiability conditions. Kumar et al.'s two supporting taxonomies: the Participant Activity Taxonomy and the Chat Trajectory Taxonomy (Tables III and IV in the source paper), sit at prompt-turn granularity within Mode 5 and complement the delegation-depth axis.

The present study complements both prior works along three axes: (i) it synthesizes 640 primary empirical studies at corpus scale rather than cataloguing UI modalities or observing a single-site sample; (ii) it orders interaction modes along a delegation-depth axis rather than a UI-modality axis or a prompt-turn axis; and (iii) it cross-references the emergent taxonomy against a supply-side capability matrix so that mode-layer findings can be read against the coverage of named capabilities. Treude & Gerosa's Section IV conclusion explicitly calls for "systematic evaluation of existing tools to analyze the frequency and usage of each interaction type"; the present SMS partially executes that call at the capability $\times$ SDLC-cell level, even though it adopts a different taxonomic axis.

2.3 Adjacent work on platform paradigms

Two adjacent surveys bound the platform-paradigm part of the scope. Ajimati et al [1] systematically review low-code and no-code development adoption and name citizen-developer contexts explicitly. Liu et al. [42] survey large language model-based agents for software engineering, overlapping with the two Wang surveys on the supply side but with a broader focus on LLM capabilities outside the agentic-programming frame. Both are used here as snowballing seeds (§3.2) in addition to the three primary reference surveys.

2.4 Methodological anchors

The study uses four methodological anchors, each cited at the point where a decision is made. Petersen et al. [52, 53] govern the systematic mapping study (SMS) design: the five-facet classification scheme, the PRISMA attrition flow, and the visualization conventions in §4.1. Cruzes & Dybå [12, 13] govern the five-step thematic synthesis that produces the RQ2 taxonomy, together with the separation of extraction (stable descriptive coding) from synthesis (derived higher-order themes). Wieringa et al. [79] govern the F1 contribution-type vocabulary and the Evaluation / Validation Research distinction used in the research agenda (§5.3). Ali & Petersen [4] govern the backward snowballing procedure. The trustworthiness audit in §6 applies Lincoln & Guba's [40] four qualitative-research criteria — credibility, confirmability, dependability, and transferability — as adapted to software engineering by Cruzes & Dybå [12].

2.5 Positioning

This SMS is therefore positioned as the first demand-side complement to a supply-side-saturated subfield. It does not compete with the three reference surveys; it maps the human-side usage distribution they do not aim to characterize. Table 4 (Appendix C) renders the four objectives $\times$ three reference surveys novelty-comparison matrix and documents zero overlap at any cell.

3 METHOD

3.1 Study design

This study is a systematic mapping study [52, 53]. SMS is appropriate here because AI-SE is an emerging subfield in which scoping, classification, and trend identification are more tractable than pooled effect estimation. Execution proceeded through seven phases - Phase 0: protocol scaffolding, Phase 1: protocol finalization and capability harmonization, Phase 2: search and screening, Phase 3: full-text extraction with concurrent open coding, Phase 4: code consolidation and theme translation, Phase 5: frequency analysis and cross-tabulation, Phase 6: trustworthiness assessment, and Phase 7: manuscript drafting. Every phase produces a tagged artefact set and a commit checkpoint; every borderline classification decision is logged in an append-only decision register with the rule applied and the rationale.

3.2 Protocol scaffolding and finalization

The study protocol was developed in two distinct phases prior to the formal execution of the search. Phase 0 (Scaffolding) involved the commitment of the initial pre-registered protocol to version control, encompassing the research questions, a Systematic Mapping Study (SMS) design outline, a preliminary codebook, an extraction form template, and six Scopus search-string archetypes.

Phase 1 (Finalization) yielded four primary protocol artifacts that served as the foundational framework for subsequent stages:

- **Five-Facet Codebook:** Operationalizes the F1–F5 vocabularies through formal definitions, positive/negative exemplars, and boundary-case rules (codebook.md).
- **Harmonized Capability Vocabulary:** A consolidated 19-capability taxonomy that collapses near-synonyms identified across the three primary reference surveys (capability_list.csv).
- **Extraction Schema:** Defines the per-paper data fields, sample characterization metrics, and capability-annotation formats (extraction_schema.md).
- **Inclusion and Exclusion Criteria (IC/EC):** Establishes the mechanical filters and manual screening heuristics (inclusion_exclusion.md; see Appendix A.2).

The operational definitions of the IC/EC and codebook boundary cases were subsequently validated through a screening pilot conducted on a convenience sample. Refinements emerging from this pilot were documented in a formal report (pilot_report.md) and integrated into the final criteria. Post-Phase 1, the protocol artifacts remained immutable; any subsequent deviations were recorded as formal entries within a centralized decision register.

3.3 Search strategy

The systematic search was conducted through a multi-stage process involving automated database queries and manual citation tracking. To ensure comprehensive coverage, six Scopus search strings were executed in parallel, targeting distinct domains: code-completion tools, conversational assistants, agentic Software Engineering (SE) tools, AI-augmented low-code/no-code platforms, visual/declarative programming, and citizen-developer contexts. The Scopus result set was complemented by a dedicated ACM Digital Library run using ACM-DL query syntax, and supplemented by backward snowballing [4] from the five seed surveys: the three reference surveys [48, 71, 73] plus two topic-adjacent papers [1, 42]. Duplicate overlap between ACM-DL and Scopus was removed at deduplication. The full search strings, per-string hit counts, and replacement decision are reproduced in Appendix A.1.

3.4 Inclusion and exclusion criteria; screening

The review protocol established five inclusion (IC) and six exclusion (EC) criteria prior to screening (Appendix A.2). To be included, papers required: (IC1) publication between 2022–2026; (IC2) empirical evidence of human interaction with AI software engineering agents, AI coding tools, or AI-enhanced low-code platforms; (IC3) peer-reviewed status; (IC4) English language; and (IC5) a minimum length of four pages. Conversely, papers were excluded if they were: (EC1) solution proposals lacking user evidence; (EC2) benchmark-only studies without human participants; (EC3) outside the software engineering domain; (EC4) secondary studies (e.g., literature reviews); (EC5) short papers, posters, or extended abstracts; or (EC6) inaccessible for full-text retrieval within two weeks.

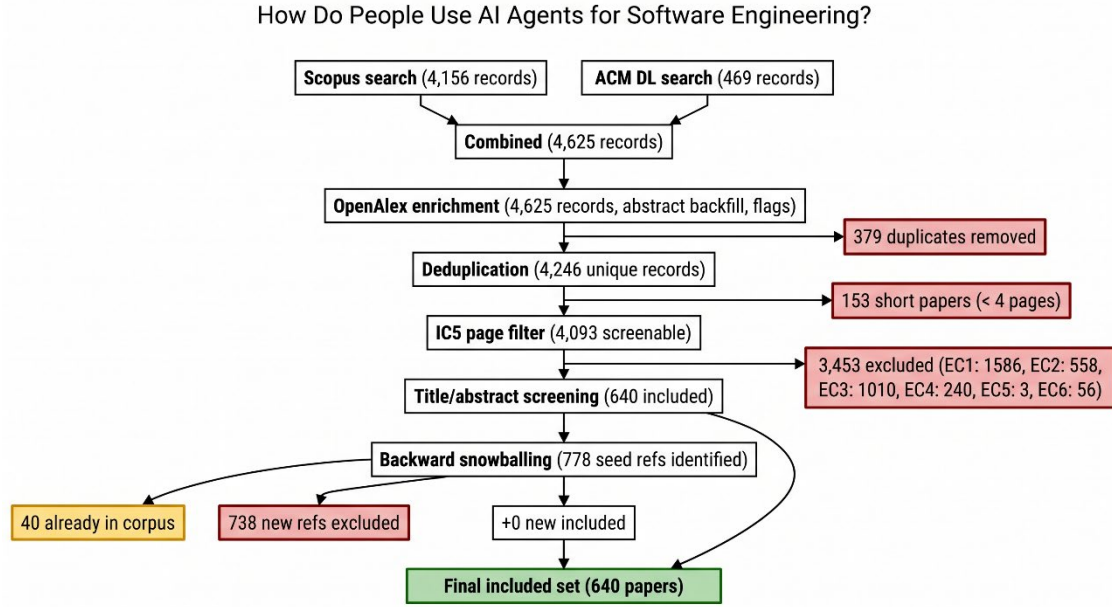


Figure 1: Illustration of the corresponding PRISMA screening flow

Figure 1 illustrates the corresponding PRISMA screening flow. The initial search yielded 4,625 records (4,156 Scopus; 469 ACM DL). After removing 379 duplicates and filtering out 153 short papers (IC5), 4,093 records advanced to title and abstract screening, resulting in 3,453 exclusions. Backward snowballing from five seed surveys yielded no new additions, as all relevant references were already captured. The final included corpus comprises 640 papers. A full reconciliation of execution-time labels against the formal criteria is provided in Appendix A.3.

3.5 Classification Scheme

Consistent with Systematic Mapping Study (SMS) guidance [52], the classification scheme defines the main extraction framework for the review. Each included paper was coded across five pre-registered facets (F1-F5). Two other protocol dimensions, Interaction Mode and Capability Category, were not used as initial screening facets. Instead, they were derived later during synthesis for RQ2 (§4.2) and RQ3 (§4.3). This separation keeps descriptive extraction distinct from higher-level interpretation [12].

- **F1 — Contribution type (RQ1).** F1 records the paper’s primary contribution using Wieringa et al.’s [79] six-class vocabulary: Evaluation Research, Validation Research, Solution Proposal, Philosophical, Opinion, and Personal Experience. F1 is single-valued. It was first coded during title-and-abstract screening and revised during full-text review when needed.

- **F2 — Research methodology** (RQ1). F2 records the empirical method used to collect or analyze human-interaction evidence. The controlled values are Experiment, Case Study, Mixed, Survey, Mining Study, Interview, and Field Study. F2 is single-valued. When a paper combines two or more methods, such as a survey plus interviews, it is coded as Mixed, with the component methods recorded in the extraction notes.
- **F3 — Population and context** (RQ1, RQ2). F3 captures who was studied and where the study took place. The population values are Professional Software Engineer, Student, Citizen Developer, OSS Contributor, and Mixed. The context values are Industry, Education, OSS, Lab, and Mixed. Studies without an identifiable human cohort, such as automated benchmark evaluations, were coded as Not Applicable (N/A). Accepted edge cases involving non-developer users were mapped to the closest category and logged in the decision register.
- **F4 — SDLC activity** (RQ1, RQ3). F4 records where in the software development lifecycle the AI tool was used. The nine values are Requirements, Design, Coding, Testing, Code Review, Debugging, CI/CD, Documentation, and Project Management. F4 is multi-valued because one paper may study AI use across several activities, such as both coding and debugging. Figures and tables using F4 therefore state their counting unit, such as distinct papers or paper-activity instances.
- **F5 — Agent / tool profile** (RQ1, RQ2). F5 describes the AI system being studied through two sub-facets. Development Paradigm is single-valued and records whether the tool is Pro-code, Low-code, or No-code. Interaction Modality is multi-valued and records how the tool exposes AI capability to users: Autocomplete, Conversational, IDE-Integrated, or Autonomous. This multi-valued coding preserves evidence from hybrid tools that combine, for example, autocomplete, chat, and IDE integration.

Because both F4 SDLC activity and F5 interaction modality are multi-valued, Appendix D provides a ranked full-text cross-tab showing how these two facets intersect in the extracted corpus.

F1 uses Wieringa et al.’s established contribution-type classification [79]. F2-F5 use study-specific controlled vocabularies, with operational definitions, positive and negative examples, and boundary-case rules reported in Appendix A.4.

The analysis also separates three related but distinct constructs. **Capability** refers to what an AI system can do, such as code generation, testing, or debugging. **Modality** refers to how the tool presents that capability to users, such as autocomplete, chat, IDE integration, or autonomous execution. **Interaction mode** refers to how users work with the tool in practice, such as accepting inline suggestions, prompting conversationally, reviewing AI output, or delegating a task. This distinction prevents tool features from being conflated with observed patterns of human-AI use. Appendix B.1 provides the full construct definitions.

3.6 Extraction and open coding

Data extraction used a structured schema to capture bibliographic metadata, F1-F5 facet codes, sample characteristics, verbatim human-AI interaction passages, and capability annotations. Capability annotations were standardized using a harmonized 19-capability vocabulary derived from the three core reference surveys. This design allowed the same corpus to support three different analyses: the RQ1 evidence landscape, the RQ2 interaction-mode synthesis, and the RQ3 capability-by-SDLC gap matrix.

Extraction followed two modes, depending on the evidence available for each paper.

- **Mode A (Full-Text Extraction):** The retrieval process recovered 258 PDFs from the 640-paper corpus. Of these, 208 papers fell within the page-count envelope and were extracted from full text. These papers

provide the strongest basis for passage-level analysis because they contain richer descriptions of tool use, participant behavior, and study context.

- **Mode B (Abstract-Only Extraction):** The remaining 432 papers were extracted from abstracts because no usable PDF was retrieved or because the PDF exceeded the page-count envelope. Mode B papers still contributed to RQ1 because F1-F5 facet values could usually be coded from title and abstract information. Where abstracts contained enough detail, they also contributed to RQ3 capability annotation and, in some cases, to RQ2.

The key methodological distinction is therefore not simply full text versus abstract, but whether a paper contained usable interaction evidence. First-pass open coding generated 1,016 in-vivo and descriptive codes across 290 passage-bearing papers: all 208 Mode A papers and 82 Mode B papers whose abstracts included quotable human-AI interaction text. This 290-paper subset forms the evidentiary base for the inductive RQ2 synthesis. The remaining 350 Mode B papers did not provide passage-level interaction evidence; they contribute to the RQ1 landscape and, where capability evidence was present, to the RQ3 matrix.

This split is treated as a transferability boundary rather than a flaw in the corpus. Full-corpus claims, such as RQ1 facet distributions, use the 640-paper denominator. Passage-level claims for RQ2 use the 290-paper grounding base. Capability-gap claims for RQ3 use the subset of papers with capability annotations and usable SDLC metadata. The results therefore report denominators explicitly wherever the unit of analysis changes.

3.7 Analysis

The analysis followed three tracks, one for each research question:

- **RQ1: Evidence landscape.** RQ1 describes the corpus at the paper level. We computed descriptive distributions for the five extraction facets (F1-F5) across the full 640-paper corpus. Because F4 SDLC activity and F5 interaction modality can be multi-valued, all figures and tables state their counting unit, such as distinct papers, paper-activity bindings, or paper-modality bindings. Following SMS guidance[52, 53], the results are presented through bar charts, heatmaps, and a bubble plot to show publication trends, methodological distribution, population coverage, SDLC coverage, and tool-profile patterns.
- **RQ2: Interaction-mode synthesis.** RQ2 uses inductive thematic synthesis [12, 13], to identify how humans and AI tools interact in practice. The analysis began with 1,016 in-vivo and descriptive open codes from the 290 passage-bearing papers. These codes were consolidated into 707 canonical labels using sentence-transformer clustering, followed by manual rater approval for all merges, splits, and label changes. The canonical labels were then classified into five interaction modes, plus a Residuals category, along the pre-committed delegation-depth axis.
During synthesis, an initial eight-mode frame was reduced to five modes. Paired Collaboration was folded into Conversational Prompting; Scaffolded Guidance was distributed across Conversational Prompting and Delegated Task Execution; and Human-in-the-Loop Delegation and Autonomous Orchestration were consolidated into Delegated Task Execution. The final mode layer reached saturation: no new modes appeared in the final 29 of the 290 passage-bearing papers. Because canonical labels are more fine-grained than modes, saturation is claimed only for the five-mode taxonomy, not for every possible label. Mode exemplars were also checked with an offline paraphrase-linting procedure to avoid excessive overlap with source passages.
- **RQ3: Capability-by-SDLC gap analysis.** RQ3 compares the supply-side capabilities emphasized in the three reference surveys with demand-side empirical evidence in the corpus. We cross-tabulated 19

harmonized capabilities against nine SDLC activities, producing a 19×9 matrix of 171 cells. Each cell counts distinct papers that contain both a capability annotation and the relevant F4 SDLC activity.

The gap rule separates three cases. A gap is a nonzero cell whose evidence count falls at or below the 25th percentile of the nonzero cell distribution. In this corpus, P25 is 2.0, so cells with one or two supporting papers are treated as low-evidence gaps. An empty cell has no supporting paper and is reported separately, because absence of evidence is different from thin evidence. All remaining cells are treated as adequately populated relative to this corpus. Sensitivity checks across the P20-P40 range assess whether the gap set is stable under alternative thresholds.

Finally, the 42 gap cells were assigned interpretive hypothesis labels: adoption lag, design gap, or organizational readiness barrier. These labels are used to structure the research agenda, not to alter the matrix itself. To preserve the boundary between extraction and synthesis [12, 13], the qualitative labels were recorded in the decision register rather than embedded as data columns in the matrix.

4 RESULTS

4.1 RQ1 — Evidence landscape

The RQ1 results show a fast-growing but uneven empirical landscape. The final corpus contains 640 studies published between 2022 and 2026, with 584 papers (91.3%) appearing from 2024 onward. The 2026 count is partial because the search cutoff occurs before year end. This recency skew reflects the rapid post-ChatGPT and post-Copilot expansion of empirical AI-SE research. The venue distribution also suggests a field still moving quickly: 456 papers (71.2%) are conference papers, compared with 184 journal articles (28.8%). Table 1 summarizes the year, contribution-type, and method distributions.

Table 1: Corpus descriptives (N = 640)

Year	Count	%	F1 Class	Count	%	F2 Method	Count	%
2022	8	1.2%	Evaluation Research	459	71.7%	Experiment	249	38.9%
2023	48	7.5%	Validation Research	143	22.3%	Mixed	139	21.7%
2024	226	35.3%	Solution Proposal	29	4.5%	Case Study	92	14.4%
2025	288	45.0%	Personal Experience	7	1.1%	Mining Study	63	9.8%
2026	70	10.9%	Opinion	2	0.3%	Survey	63	9.8%
						Interview	20	3.1%
						Field Study	14	2.2%

F1 confirms that the corpus is strongly empirical. Evaluation Research accounts for 459 papers (71.7%), and Validation Research accounts for another 143 (22.3%). Together, these two categories represent 94.0% of the corpus. This distribution is consistent with the study’s demand-side focus: the retained literature is primarily concerned with evaluating AI-SE tools, practices, or effects, rather than proposing unevaluated systems or offering opinion-based accounts.

F2 shows that the empirical base is broad but methodologically uneven. Experiments are the dominant design (249 papers, 38.9%), followed by mixed-method studies (139, 21.7%) and case studies (92, 14.4%). By contrast, interviews (20, 3.1%) and field studies (14, 2.2%) are rare. This imbalance matters because many questions about AI-SE adoption, trust, oversight, and workflow fit are difficult to answer through controlled experiments alone. Figure 2 2 therefore highlights not only what methods are common, but also where the corpus is thin: in situ and first-person accounts of practice remain underdeveloped.

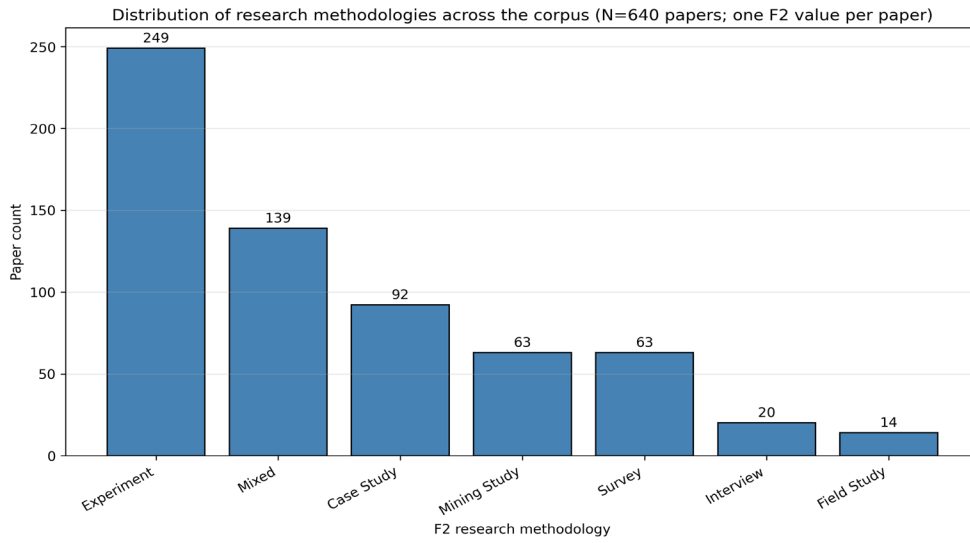


Figure 2: F2 research-methodology distribution for the 640-paper corpus.

F3 shows partial visibility into who is being studied. Of the 640 papers, 394 (61.6%) report a canonical population value, while 246 (38.4%) are missing or not applicable for population analysis. Within the populated subset, students, professional software engineers, and mixed cohorts appear at broadly comparable levels. Citizen developers and OSS contributors, however, are rarely represented, with only 19 and 3 papers respectively. This imbalance matters because citizen developers are central to the broader AI-SE adoption narrative, yet they remain weakly visible in the empirical corpus. Figure 3 makes this sparsity visible across years and SDLC activities.

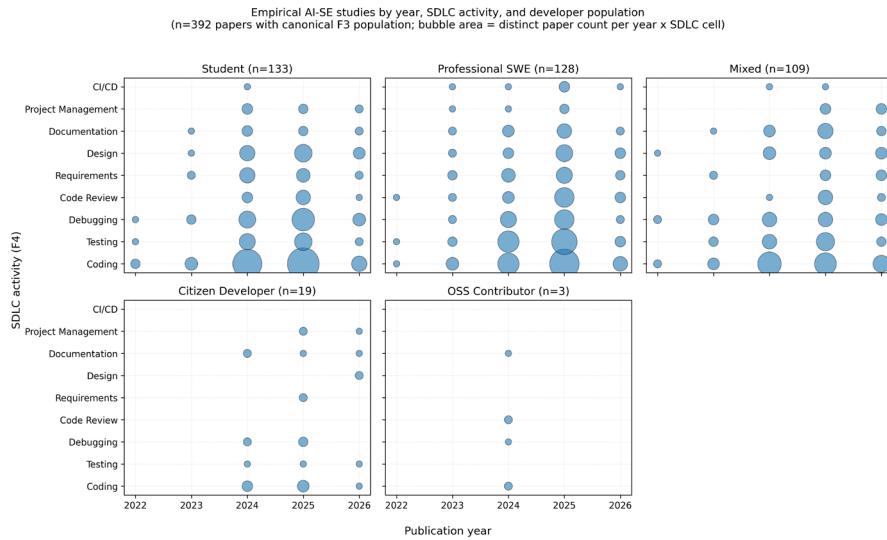


Figure 3: Year $\times$ SDLC activity $\times$ developer population bubble plot ($n = 392$ of the 394 canonical-F3 papers; 2 Mixed-population papers omitted because no F4 SDLC activity could be assigned). Bubble area = distinct paper count per year $\times$ SDLC cell.

F4 shows that empirical attention remains concentrated around code-centered work. The 640 studies produce 1,083 paper-SDLC activity bindings. Coding, Testing, and Debugging account for 705 of these bindings (65.1%), while

Requirements, Project Management, and CI/CD are much thinner. This pattern suggests that empirical AI-SE research still follows the traditional coding-assistant center of gravity, even as supply-side surveys increasingly describe lifecycle-spanning and agentic capabilities. Figure 4 shows that this code-centered structure persists across publication years.

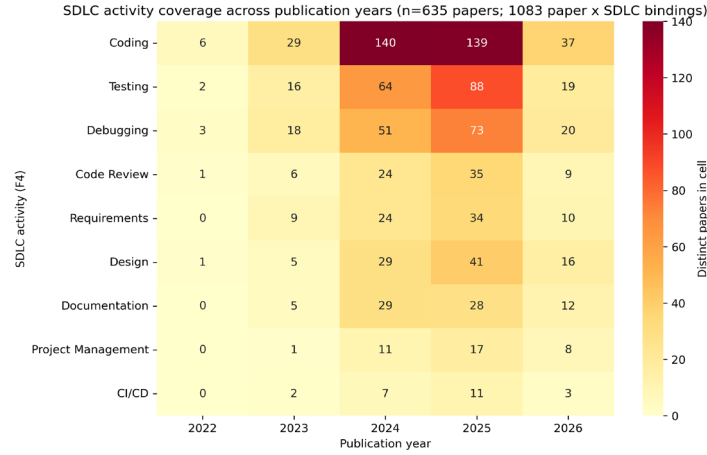


Figure 4: SDLC × year evidence heatmap. Each cell is the distinct-paper count for the given SDLC activity and publication year. Coding, Testing, and Debugging dominate; Requirements, Project Management, and CI/CD are thin across every year.

F5 shows a similar concentration at the tool-paradigm level. Pro-code tools dominate the corpus, with 538 papers (84.1%), compared with 72 no-code papers (11.3%) and 30 low-code papers (4.7%). This does not mean low-code and no-code AI tools are unimportant; rather, it means that published empirical evidence about them is comparatively sparse. As a result, paradigm-specific claims, especially those involving low-code and no-code settings, should be treated as directional rather than definitive. Figure 5 reports the paradigm and modality distribution; its stacked modality view includes 639 papers because one record lacks a plottable modality value.

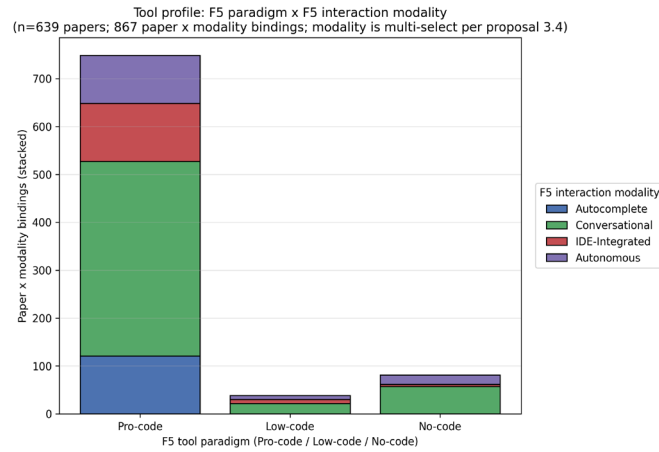


Figure 5 F5 tool profile. Paradigm (Pro-code, Low-code, No-code) on the horizontal axis; interaction-modality stacks (Autocomplete, Conversational, IDE-Integrated, Autonomous) within each paradigm bar.

Taken together, RQ1 establishes the empirical baseline for the rest of the study. The corpus is recent, empirical, and substantial, but it is also concentrated around pro-code tools, code-centered SDLC activities, experimental designs, and familiar participant populations. These asymmetries provide the context for the RQ2 interaction-mode

taxonomy and help explain the capability-by-SDLC evidence gaps reported in RQ3. Appendix D provides a closer ranked full-text view of how F4 SDLC activities intersect with F5 interaction modalities.

4.2 RQ2 — Human–AI Interaction Mode Taxonomy

The Delegation-Depth Axis. RQ2 examines how human-AI interaction is described in the passage-bearing subset of the corpus. We organize the resulting taxonomy along a **delegation-depth axis**, defined as the distribution of planning, execution, and verification responsibilities between the human and the AI. At one end, the human retains close control over all three functions. At the other end, the AI performs larger units of work while the human mainly scopes the task and verifies the result. The axis was specified in the protocol, but the five modes placed along it were derived inductively from the empirical data.

Five interaction modes. The synthesis of 707 canonical labels produced five interaction modes and a Residuals category, as summarized in Table 2. The five modes represent distinct patterns of human-AI work: inline suggestion acceptance, conversational prompting, visual or declarative composition, review and validation, and delegated task execution. Residuals are not a sixth mode. They capture evidence about interaction, such as outcomes, constraints, affordances, or meta-observations, where the passage did not describe a mode-defining interaction episode.

Table 2: Five-mode interaction taxonomy summary

#	Mode	Canonical labels	Share (%)	Saturation
1	Inline Completion	4	0.6%	Stable; corpus-bias caveat
2	Conversational Prompting	69	9.8%	Stable
3	Visual / Declarative Composition	9	1.3%	Stable; paradigm-skew caveat
4	Review & Validation	82	11.6%	Stable
5	Delegated Task Execution	75	10.6%	Stable
r	Residuals (outcome / constraint / meta / affordance)	468	66.2%	Sub-structured

The first pattern is that explicit mode evidence is narrower than the broader empirical discussion around AI-SE use. Residuals account for 468 of 707 canonical labels. These labels divide into four sub-layers: outcomes (192), constraints (119), meta-observations (101), and affordances (56). This shows that many empirical studies report what happened around AI use, such as benefits, barriers, risks, or tool affordances, without describing the interaction pattern in enough detail to classify it as a mode. The taxonomy therefore reports both the five interaction modes and the Residuals sub-structure, but treats them as different empirical signals.

The second pattern is that Mode 1 is thin in the corpus but should not be read as rare in practice. Inline completion is represented by only four canonical labels, far below the pre-registered re-open trigger. This likely reflects publication visibility rather than usage sparsity. Inline completion has been widely deployed since 2021 and may no longer appear as a novel empirical object in many venues. In addition, short reports of completion-style tool use were often excluded because they did not meet the empirical or length criteria. We therefore treat Mode 1 prevalence claims with caution.

The third pattern is that code-centered activity persists across the delegation axis. In the passage-bearing subset, Coding is the leading SDLC activity for every mode row: Mode 1 (80%), Mode 2 (60%), Mode 3 (82%), Mode 4 (53%), Mode 5 (58%), and Residuals (61%). At the same time, the mode-specific variation is meaningful. Mode 4 has the strongest Code Review share (25.5%), consistent with its evaluative function. Mode 5 has the strongest CI/CD footprint (11 papers, 9.6%), consistent with delegated execution extending into operational workflows.

The fourth pattern concerns tool paradigm. Pro-code tools dominate across the delegation axis: 253 of the 290 passage-bearing papers with F5 paradigm values are Pro-code (87.2%), compared with 26 No-code (9.0%) and 11 Low-code (3.8%). Mode 3 is the main exception. Its low-code share is 18.2% of its 11-paper subsample, higher than

in any other mode, which is consistent with the visual and declarative character of low-code environments. No-code evidence is not concentrated in one mode; it appears across Modes 2, 3, 4, and 5.

Population and context patterns remain thin outside familiar settings. Professional SWE / Industry and Student / Education are the most visible population-context cells in the passage-bearing subset, while Citizen Developer and OSS Contributor evidence is sparse. Together, Citizen Developer and OSS Contributor populations account for only 7 of 255 papers with usable population-context variation data. These cells are too small to support mode-level prevalence claims, so industry-versus-education and citizen-developer comparisons remain important topics for future empirical work.

Finally, the mode layer reached saturation. No new modes emerged in the final 29 of the 290 passage-bearing papers. This supports the stability of the five-mode taxonomy within the corpus. Canonical-label saturation was not reached, which is expected because canonical labels are finer-grained than modes. The dependable claim is therefore the five-mode partition, not an exhaustive inventory of every possible interaction label.

4.3 RQ3 — Capability–evidence gap matrix

Research Question 3 examines whether the capabilities emphasized in supply-side AI-SE surveys are matched by demand-side empirical evidence. To do this, we cross-tabulated 19 harmonized capabilities against nine SDLC activities, producing a 19×9 matrix with 171 capability-activity intersections. Each cell counts distinct papers that report a given capability in each SDLC activity.

The matrix is based on 776 capability annotations from 384 papers. After excluding annotations without usable F4 SDLC metadata, 383 papers populate the final matrix. This distinction matters because RQ3 is not a full-corpus census of all 640 papers; it is a capability-evidence analysis over the subset of papers with identifiable capability annotations and SDLC activity codes.

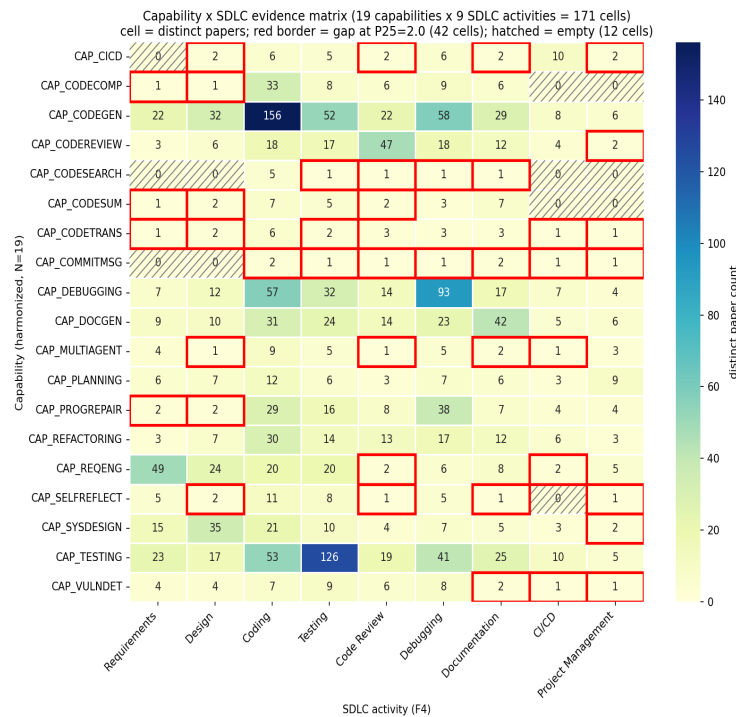


Figure 6: Capability × SDLC evidence matrix (19 capabilities × 9 SDLC activities = 171 cells)

Applying the gap rule defined in §3.7, the 171 cells are divided into three groups: 117 adequately populated cells, 42 low-evidence gap cells, and 12 empty cells. A gap is a nonzero cell with evidence at or below the 25th percentile of the nonzero cell distribution. In this corpus, P25 is 2.0, so gap cells contain one or two supporting papers. Empty cells are reported separately because they represent no observed evidence, not merely thin evidence. This distinction prevents under-studied intersections from being conflated with untouched ones.

The matrix shows a strong diagonal pattern. Evidence is densest where capabilities are applied to their most natural SDLC activity: Code Generation in Coding (156 papers), Testing in Testing (126), and Debugging in Debugging (93). Off-diagonal evidence is much thinner. The 54 low-evidence or empty cells cluster especially around early-lifecycle work, operational work, and coordination work, including Requirements, Design, CI/CD, and Project Management. Figure 6 visualizes this asymmetry, with red borders marking low-evidence gaps and hatching marking empty cells.

The highest-priority gaps are not obscure capabilities. Table 6 shows that the top-ranked cells involve capabilities emphasized across the reference surveys, but weakly evidenced outside their familiar coding-centered contexts. Code Completion is thin in Requirements and Design; Code Translation is thin in Requirements, CI/CD, and Project Management; and Self-Reflection and Iterative Refinement is thin in Code Review, Documentation, and Project Management. Lower-ranked but still high-priority cells extend the same pattern to Multi-Agent Collaboration and Vulnerability Detection.

Table 3: Top-13 prioritized capability $\times$ SDLC gaps (priority ≥ 1.00)

Rank	Mode	Label	SDLC Activity	Evidence	Priority	ERP3 Buket
1	CAP_CODECOMP	Code Completion	Design	1	1.50	design_gap
2	CAP_CODECOMP	Code Completion	Requirements	1	1.50	design_gap
3	CAP_CODETRANS	Code Translation	CI/CD	1	1.50	organizational_readiness_barrier
4	CAP_CODETRANS	Code Translation	Project Management	1	1.50	organizational_readiness_barrier
5	CAP_CODETRANS	Code Translation	Requirements	1	1.50	design_gap
6	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Code Review	1	1.50	adoption_lag
7	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Documentation	1	1.50	adoption_lag
8	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Project Management	1	1.50	adoption_lag
9	CAP_MULTIAGENT	Multi-Agent Collaboration	CI/CD	1	1.00	adoption_lag
10	CAP_MULTIAGENT	Multi-Agent Collaboration	Code Review	1	1.00	adoption_lag
11	CAP_MULTIAGENT	Multi-Agent Collaboration	Design	1	1.00	design_gap
12	CAP_VULNDET	Vulnerability Detection	CI/CD	1	1.00	organizational_readiness_barrier
13	CAP_VULNDET	Vulnerability Detection	Project Management	1	1.00	organizational_readiness_barrier

The gap categories provide a caution interpretation of what may be missing from the empirical literature. Of the 42 gaps, 22 are coded as organizational readiness barriers, 14 as design gaps, and 6 as adoption lags. These labels are hypotheses for future research, not matrix data. Their distribution suggests that many sparse cells may reflect integration, governance, accountability, and workflow-readiness challenges rather than the absence of technical capability alone.

The SDLC distribution reinforces this interpretation. CI/CD is the most visibly under-evidenced column, with five gap cells and four empty cells. Requirements, Design, and Project Management together account for 18 of the 42 gap cells. This pattern indicates a structural imbalance: empirical AI-SE research remains concentrated around coding, testing, and debugging, while upstream planning and downstream operational coordination receive much thinner evidence.

The gap set is robust under sensitivity checks. The P25 gap set contains 42 cells; the same 42 cells remain gaps at P20, P33, and P40. Relaxing the threshold adds cells, producing 53 gaps at P33 and 72 at P40, but no original P25 gap exits the set. Higher thresholds should therefore be read as advisory expansions, not reversals of the primary result.

Finally, paradigm-specific gap analysis is limited by corpus sparsity. The low-code submatrix contains 141 empty cells out of 171, and the no-code submatrix contains 123 empty cells. In both subsets, the P25 threshold collapses to 1.0 because so few nonzero cells are available. As a result, per-paradigm gap counts should be treated as directional signals rather than standalone findings.

5 DISCUSSION

5.1 Dominant pattern: supply-saturation versus demand-under-evidence

The Supply-Demand Disconnect. A central theme common among all three research questions is the asymmetry between the saturation of supply-side capabilities and the scarcity of demand-side empirical evidence. While the core reference surveys [48, 71, 73] detail an extensive array of theoretical capabilities (harmonized here into 19 categories), empirical validation remains highly uneven. Of the 171 capability-by-SDLC intersections, nearly a third lack sufficient backing: 42 are classified as under-evidenced gaps, and 12 are entirely empty.

Asymmetry in Human-AI Interaction Modes. This supply-demand asymmetry persists at the Human-AI interaction mode layer (see §4.2). For instance, Mode 5 (Delegated Task Execution), that is identified by reference surveys as the most actively developed technological frontier, accounts for just 10.6% (75 of 707) of the canonical labels in the empirical corpus. Conversely, Mode 1 (Inline Completion), despite ubiquitous mainstream deployment since 2021, represents a mere 0.6% (4 labels); however, this reflects established publication bias against mature tools rather than an absence of practitioner utility. Ultimately, as detailed in the novelty-comparison matrix (Appendix C.1), the reference surveys map the supply side of the AI-SE landscape, whereas this Systematic Mapping Study (SMS) systematically audits the demand side.

Complementarity and Methodological Novelty. Our findings complement, rather than overlap with, Treude and Gerosa’s [66] UI-modality taxonomy. While their framework catalogs interaction modalities at the mechanical user-interface level, this SMS scales interaction modes along an inductively derived delegation-depth axis, which is then cross-referenced against the supply-side capability matrix. To our knowledge, this specific methodological synthesis, i.e. combining a strictly defined statistical cross-tabulation (the 25th-percentile threshold) with a Wieringa-typed prioritization to structure a pre-registered research agenda—has no direct precedent in the AI-SE mapping-study literature.

5.2 Practical implications

The findings of this SMS study yield actionable insights for organizations, tool builders, researchers, and educators.. In each domain, strategic priorities are derived directly from the observed asymmetrical differences between concentrated empirical evidence and identified research gaps.

For **Organizations**. The delegation-depth taxonomy can be used as an adoption and governance instrument. Rather than asking whether a team is “AI-ready” in general, organizations can map current practice by mode: inline assistance, conversational prompting, visual/declarative composition, review and validation, or delegated task execution. This creates a more precise view of operational risk. Mode 2 adoption requires prompt and context-management skills; Mode 4 requires review protocols and accountability norms; Mode 5 requires checkpointing, rollback, audit trails, and explicit ownership of final decisions. The corpus suggests that organizational value depends less on introducing AI everywhere than on matching delegation depth to task criticality. Park [50] shows that continued use among industry developers is shaped by perceived usefulness and low workflow disruption, especially when tools integrate naturally into IDEs. Weisz et al. [76] similarly show that enterprise developers often review, modify, or repurpose AI outputs

rather than accepting them unchanged. These findings support a staged adoption model: use low-risk, low-accountability tasks to build fluency; introduce review-heavy workflows where human judgment is visible; and reserve delegated execution for settings with clear checkpoints and responsibility boundaries. Vigh et al. [70] further show why this matters: practitioners deliberately avoid AI in security-critical workflows and remain aware that accountability cannot be transferred to the model..

For **Tool Builders**, the gap matrix points to integration problems, not only capability problems. Many high-priority gaps involve mature capabilities, such as code completion, code translation, self-reflection, multi-agent collaboration, and vulnerability detection, appearing outside their usual coding-centered settings. This suggests that vendors should prioritize workflow fit, governance support, and reviewability over simply adding more generation features. For example, code translation in CI/CD or project management is less a model-only problem than a coordination problem: teams need traceability, intent preservation, testing hooks, and clear handoff points. The local corpus reinforces this interpretation. Tambon et al. [63] show that LLM-generated code can contain bugs that are non-trivial to diagnose and repair. Epperson et al. [16] show that multi-agent systems require steering, state reset, and checkpointing support because failures emerge from both individual agents and their interactions. Tufano et al. [68] show that AI is already appearing in process-oriented OSS workflows, including commit messages, PR descriptions, and CI/CD review, but also surfaces hallucination risks. For tool builders, the implication is clear: higher-delegation tools need interfaces for inspection, correction, provenance, and recovery, not only more autonomous execution.

For **Researchers**. The corpus is large but methodologically uneven. Experiments provide much of the current evidence, while interviews and field studies remain scarce. As a result, the literature is stronger at measuring task-level performance than at explaining how teams adopt AI tools, govern their use, calibrate trust, manage tacit repository knowledge, and absorb review burden in everyday work. Future studies should therefore prioritize in-situ research on Mode 4 and Mode 5 interaction, particularly in requirements, design, CI/CD, documentation, project management, low-code, and no-code contexts. Controlled experiments remain valuable, but they should be complemented by longitudinal field studies, interviews, and mixed-method designs that capture organizational constraints and workflow adaptation over time. This need is consistent with Gao et al. [23] and Ouyang et al. [49], who respectively highlight capability-side gaps and replication challenges caused by LLM non-determinism. The research agenda in §5.3 should therefore be read as a prioritized starting point, not a closed list; future qualitative work may surface additional priorities not visible in the present gap matrix.

For **Educators**. While student-population studies constitute the largest demographic subset (133 papers), the majority evaluate a single tool (e.g., GitHub Copilot or ChatGPT) within a localized course setting [69]. Critical vulnerability is identified by Bo et al. [9], who document a Mode-5 failure mode where sycophantic LLM outputs actively mislead novice problem-solvers. This underscores that delegated-task assignments necessitate explicit checkpoint design embedded at the curriculum level. The interaction taxonomy provides a blueprint for a capability-ladder syllabus: establishing Mode 1 fluency in introductory courses, Mode 2 prompting in mid-level programs, and Mode 4 evaluative exercises before introducing Mode 5 delegation tasks. This progression aligns educational frameworks with the empirical trajectory of professional practice rather than transient tool popularity. Caveat: Institutional contexts may require reordering this ladder. Furthermore, comparative analyses between industry and educational deployments remain sparse in the corpus and are marked as an ongoing research priority.

5.3 Research agenda

To operationalize the identified 42 gaps and 12 empty cells, this paper proposes a structured research agenda comprising seven distinct Calls to Action. Calls 1 to 6 are corpus-internal and strictly aligned with Wieringa’s [79] taxonomy (Evaluation and Validation Research) to ensure each Call carries a pre-specified methodological class and operational entry condition. Within this subset, Calls 1–5 formulate concrete primary-study proposals targeting the 13

highest-priority gaps (priority ≥ 1.00). Call 6 operates as an integrative Evaluation Research initiative designed to address a specific cluster of ten moderate-priority gaps (priority = 0.50) centered on knowledge management.

A final directive, Call 7, extends the agenda beyond the internal gap matrix to confront the transferability boundary established by the corpus's Pro-code dominance. Rather than targeting an isolated matrix gap, Call 7 proposes a downstream taxonomy construction and validation study. Methodologically, it prescribes Evaluation Research grounded in a foundational Multivocal Literature Review [24]. Comprehensive research-question templates and population-fit guidelines for all seven Calls are provided in Appendix C.5.

1. **Call 1: AI Code Completion in Early-Lifecycle Activities** (Evaluation Research; Two priority 1.50 gaps). How do practitioners utilize code-completion tools during requirements elicitation and software design, and how do these usage-patterns diverge from the coding-stage applications that currently dominate the literature?
2. **Call 2: AI Code Translation Beyond the Coding Boundary** (Validation at Requirements; Evaluation at CI/CD and Project Management; Three priority 1.50 gaps). How do legacy-modernization initiatives integrate AI code translation into migration project management and CI/CD pipelines? Furthermore, under what conditions does automated translation reliably preserve specification intent?
3. **Call 3: Agentic Self-Reflection Outside of Coding** (Validation Research; Three priority 1.50 gaps). To what extent does agentic self-reflection yield measurably better outcomes for code-review synthesis, documentation generation, and project-management task decomposition compared to single-pass baselines? Following Kumar et al. [36], downstream studies must pre-register a falsifiable constraint: self-reflection is credited only if its quality advantage persists under user-disagreement conditions, strictly controlling for known sycophancy and overconfidence failure modes.
4. **Call 4: Multi-Agent Orchestration at Under-Studied SDLC Intersections** (Validation Research; Three priority 1.00 gaps). How effectively do multi-agent frameworks perform as collaborative code-review committees, CI/CD pipeline coordinators, and design-stage ideation partners, and which coordination patterns exhibit the greatest robustness? Downstream studies should pre-register two falsifiable metrics from Kumar et al. [36]: the unsolicited-agent-action rate (per prompt-turn) as a measure of coordination quality, and the success differential between one-shot and incremental coordination patterns across these three intersections.
5. **Call 5: AI Vulnerability Detection Beyond Coding** (Evaluation Research; Two priority 1.00 gaps). How is AI-augmented vulnerability detection embedded within CI/CD pipelines and enterprise security management? Additionally, what organizational factors best predict the effective triage of AI-generated security alerts?
6. **Call 6: Knowledge Management in Non-Native Activities** (Evaluation Research; Ten priority 0.50 gaps). Outside the primary coding flow, when and how do practitioners invoke AI-driven code search, summarization, and commit-message generation? Crucially, what marginal value or friction do these capabilities introduce at critical decision points in code review, testing, debugging, documentation, and CI/CD?
7. **Call 7: A taxonomy of AI-agent architectures beyond the Pro-code** (Evaluation Research preceded by a Multivocal Literature Review [24]; no priority-ranked gap, motivated by transferability boundary and by practitioner-reported scenarios outside the corpus). AI-agent tools vary along an architecture-type dimension, independent of the delegation-depth axis of §4.2. The Call asks two linked questions. First, what are the distinct architectural types in practice? Second, for each type, who is the typical user (developer, citizen developer, researcher-builder, or individual end user), what kind of need triggers its use (a business-process need or a software-development-lifecycle need), and does the tool bridge multiple development

paradigms (Pro-code, Low-code, No-code)? The five candidate types, each anchored where possible by an A*-ranked corpus exemplar, are: (i) generalist AI authoring IDEs (e.g., GitHub Copilot, Cursor), anchored by Vaithilingam et al. [69] and Park [50]; (ii) horizontal workflow orchestration tools with AI (e.g., n8n, Zapier), anchored by Cheng et al.'s Prompt Sapper [11] and Wu et al.'s AI Chains [80]; (iii) vertical SaaS suites with embedded AI (e.g., Salesforce Einstein, SAP Joule), anchored by Jiang et al.'s SiriusBI [29] and Weisz et al.'s watsonx Code Assistant audit [76]; (iv) agent-construction frameworks and harnesses (e.g., LangGraph, CrewAI), anchored by Epperson et al.'s AGDebugger / AutoGen / Magentic-One study [16]; and (v) deployed personal-AI-assistant platforms (no A*-ranked corpus exemplar yet; this absence is itself part of the empirical signal that motivates the MLR's grey-literature scope). These five are hypotheses to be tested, not pre-committed conclusions; the final count is determined empirically by the MLR and a companion practitioner survey [33, 34].

5.4 Pre-registered survey items for the downstream stage (ERP-3)

To empirically support the seven research calls defined in §5.3, the downstream research phase will administer a targeted practitioner survey [33]. The survey instrument is structured to validate both the architectural typology (Call 7) and the specific capability-by-SDLC gaps (Calls 1–6). The first three items prioritize Call 7, as the concurrent Multivocal Literature Review (MLR) [24] will field-test the architecture typology alongside survey piloting. Items 4–9 directly operationalize Calls 1–6. Comprehensive scoping, cognitive piloting, and survey administration are deferred to the execution phase of ERP-3.

The foundational survey item bank includes:

- **Item 1: Architectural Self-Classification (Call 7).** Practitioners are asked to map their primary tool to a candidate architectural class: generalist AI authoring IDEs (e.g., Cursor, Copilot), horizontal workflow orchestrators (e.g., n8n, Zapier), vertical SaaS with embedded AI (e.g., SAP Joule, Salesforce Agentforce), agent-construction frameworks (e.g., LangGraph, CrewAI, OpenHands [76], or deployed personal AI assistants. An open-ended option captures unclassified tools.
- **Item 2: Typology Validation (Call 7).** Evaluates the construct validity of the MLR-derived architecture typology by asking practitioners if the proposed categories, criteria, and exemplars accurately reflect their operational reality, while identifying any missing classifications.
- **Item 3: Role-Bridging and Handoff Dynamics (Call 7).** Investigates cross-paradigm workflows by identifying the tool selected when business-process requirements trigger traditional software engineering tasks. It captures the user's self-identified role (e.g., professional developer, citizen developer, researcher) and the precise point of handoff to engineering teams.
- **Item 4: Delegation-Depth Distribution (Calls 1, 4, 6).** Utilizes a matrix design to map the frequency of AI usage across the five established interaction modes (Inline Completion through Delegated Task Execution) against specific SDLC activities during a typical development sprint.
- **Item 5: Baseline Interaction Frequency.** Quantifies the proportion of coding time dedicated strictly to inline completion (Mode 1) and the frequency of accepting completions without initiating separate conversational turns, providing empirical grounding against the identified publication bias.
- **Item 6: Translation in Non-Native Contexts (Call 2).** Solicits empirical examples and friction points encountered when integrating AI code translation outside of core coding, specifically within upstream requirements, CI/CD pipelines, or legacy migration workflows.
- **Item 7: Agentic Self-Reflection and Deployment Barriers (Call 3).** Audits the deployment of iterative-refinement loops in code review, documentation, and project management. Crucially, it asks non-adopters to categorize their primary implementation blockers as technical, organizational, or trust-related.

- **Item 8: Security Alert Triage in Pipelines (Call 5).** Determines organizational accountability for triaging AI-generated security alerts within CI/CD pipelines and measures the ratio of actioned versus dismissed alerts to assess operational friction.
- **Item 9: Low-Code/No-Code Interaction Mechanics** (Transferability Boundary IV; Calls 1, 6). Asks practitioners in low-code or no-code environments to characterize their primary AI interaction mode, the specific UI mechanics of their toolset, and its corresponding architectural class from Item 1.

6 THREATS TO VALIDITY

We assess trustworthiness using the four qualitative-research criteria adapted for software engineering by Cruzes and Dybå [12, 13]: credibility, confirmability, dependability, and transferability. The audit trail includes the protocol, decision register, generated analysis files, metadata records, and the SMS reporting checklist.

Credibility. The main risk is misclassification during screening, extraction, or synthesis. To reduce this risk, the study defined operational rules before analysis, including the F1-F5 facet vocabularies, the distinction between capability, modality, and interaction mode, the paraphrase-checking procedure for exemplars, and the P25 rule for identifying low-evidence gaps. RQ2 claims are traceable from interaction mode to canonical label, pass-1 code, passage identifier, and paper identifier. RQ3 claims are traceable from each gap cell to the corresponding capability, SDLC activity, evidence count, and decision-register rationale.

Confirmability. The main consideration is that results could depend on hidden or informal analyst judgment. To address this, transformed outputs are retained as auditable artifacts, and generated CSV files are paired with metadata describing inputs, scripts, and timestamps. The 42 RQ3 gap cells are also linked to decision-register entries, preserving a bidirectional connection between the gap matrix and the interpretive rationale used in the research agenda.

Dependability. The main risk is instability in the inductive taxonomy or sensitivity to the selected gap threshold. For RQ2, mode-layer saturation was reached: no new interaction modes appeared in the final 29 of the 290 passage-bearing papers. This supports the stability of the five-mode taxonomy within the corpus, although it does not imply saturation of every fine-grained canonical label. For RQ3, threshold checks show that the 42-cell P25 gap set is preserved across the P20-P40 sensitivity band. Higher thresholds add cells but do not reverse the primary gap set.

Transferability. The findings should be interpreted within the boundaries of the study design. The corpus covers peer-reviewed, English-language empirical papers from 2022 to the April 2026 search cutoff, retrieved from Scopus, ACM DL, and backward snowballing from seed surveys. Grey literature is excluded. The evidence base is also dominated by pro-code tools, while low-code and no-code studies are comparatively sparse. Population coverage is incomplete, with citizen developers and OSS contributors underrepresented. RQ2 is further limited to the 290 passage-bearing papers, not the full 640-paper corpus. Mode 1 is thin in the retained literature, likely reflecting publication visibility rather than low practitioner use. Finally, the research agenda should be read as a structured prioritization derived from the map, not as a validated account of practitioner demand.

Single-rater study. This SMS was executed by a single rater, with LLM assistance used mainly as a labeler-proposer at the screening and open-coding stages; the rater approved, renamed, merged, or split every decision, and every borderline decision is logged in the decision register. An inter-rater reliability pilot is explicitly deferred to the downstream qualitative practitioner-interview study, where practitioner evidence also serves as external validation of the RQ2 taxonomy.

7 CONCLUSION

This systematic mapping study examined the demand side of empirical AI-SE research: how AI coding assistants, software engineering agents, and AI-augmented low-code/no-code platforms are reported as being used in software

development work. While recent supply-side surveys [48, 71, 73] describe what these systems can do, this study maps where empirical evidence of human use has accumulated.

Across 640 primary empirical papers published between 2022 and the April 2026 search cutoff, the study makes three contributions. First, it provides a five-facet evidence landscape showing that the corpus is recent, code-centered, and dominated by pro-code tools. Second, it derives a five-mode Human-AI Interaction Mode Taxonomy organized along a delegation-depth axis, from inline completion to delegated task execution. Third, it constructs a 19×9 capability-by-SDLC matrix that compares harmonized supply-side capabilities with demand-side empirical evidence, identifying 42 low-evidence gaps and 12 empty intersections.

The main finding is a supply-demand asymmetry. AI-SE capability inventories are broad and fast-moving, but empirical evidence remains concentrated around coding, testing, and debugging. Evidence is thinner for requirements, design, documentation, CI/CD, project management, low-code, and no-code contexts. This does not mean that every sparse intersection is equally meaningful. Some are plausible but under-studied, such as self-reflective agents in code review or vulnerability detection in CI/CD. Others may be structurally weak under the study’s definitions. Distinguishing low-evidence gaps from empty or weakly applicable intersections is therefore essential for interpreting the map.

The study also shows that capability, modality, and interaction mode should be treated as separate constructs. A tool may expose several modalities, support multiple capabilities, and still be used through different interaction modes depending on task and context. The delegation-depth taxonomy makes this distinction visible and provides a vocabulary for comparing studies that otherwise use inconsistent labels such as assistant, copilot, chatbot, and agent.

For practice, the findings suggest that organizations should evaluate AI adoption by the level of delegation they are prepared to govern, not only by tool availability. Tool builders should look beyond saturated coding-stage features toward planning, review, documentation, pipeline, and coordination workflows. Researchers should prioritize field studies, interviews, and longitudinal evaluations in under-evidenced SDLC contexts. Educators can use the taxonomy to scaffold AI-SE instruction from low-delegation assistance toward higher-delegation tasks that require explicit review and accountability.

Future research should validate these patterns in practitioner settings, especially where the present corpus is thinnest: low-code, no-code, requirements, design, documentation, CI/CD, and project management contexts. A key next step is to study how AI-SE and agentic-AI capabilities transfer beyond coding. This includes examining how agents support requirements elicitation and traceability, design exploration and architecture decision-making, documentation generation and maintenance, CI/CD monitoring and release coordination, and project-management tasks such as planning, estimation, triage, and progress tracking. Such studies should evaluate not only task performance, but also human oversight, accountability, verification practices, integration with existing tools, and the conditions under which delegation improves or harms software engineering work. The study’s evidence landscape, interaction taxonomy, gap matrix, and research agenda provide a reproducible foundation for this work.

Acknowledgments

The author would like to thank Professor Ta and Professor Zhu for their expert supervision, constructive feedback, and dedicated mentorship throughout the lifecycle of this project, which significantly shaped the direction of this research.

Declaration of Generative AI Use

During the preparation of this work, the author used generative AI tools to support selected research and writing tasks. Gemini 3.1 Pro was used to assist with structured paper extraction, qualitative coding and label proposal, taxonomy-classification support, manuscript drafting, and figure generation. GPT-5.4 was used as an independent verifier to check extraction quality and validate analytical outputs. Claude Code was used as part of the repository execution

environment to support code development, workflow implementation, and reproducibility checks. The author reviewed, verified, edited, and approved all AI-assisted outputs and takes full responsibility for the accuracy, integrity, and final content of the submitted manuscript.

REFERENCES

- [1] Matthew Oladeji Ajimati, Noel Carroll, and Mary Maher. 2025. Adoption of low-code and no-code development: A systematic literature review and future research agenda. *Journal of Systems and Software* 222, (April 2025), 112300. <https://doi.org/10.1016/j.jss.2024.112300>
- [2] Adam Alami, Victor Jensen, and Neil Ernst. 2025. Accountability in code review: The role of intrinsic drivers and the impact of llms. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3721127>
- [3] Farman Ali, Awais Ahmed, Mohammad Amin Alipour, and Hugo Terashima-Marin. 2025. Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model. *Journal of Computers in Education* (2025). <https://doi.org/10.1007/s40692-025-00375-w>
- [4] Nauman Bin Ali and Kai Petersen. 2014. Evaluating strategies for study selection in systematic literature studies. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, September 18, 2014. ACM, Torino Italy, 1–4. <https://doi.org/10.1145/2652524.2652557>
- [5] Aylton Almeida, Laerte Xavier, and Marco Tulio Valente. 2024. Automatic library migration using large language models: First results. In *Proceedings of the 18th ACM / IEEE international symposium on empirical software engineering and measurement (ESEM '24)*, 2024. . <https://doi.org/10.1145/3674805.3690746>
- [6] Eman Abdullah AlOmar, Anushkrishna Venkatakrishnan, Mohamed Wiem Mkaouer, Christian Newman, and Ali Ouni. 2024. How to refactor this code? An exploratory study on developer-ChatGPT refactoring conversations. In *Proceedings of the 21st international conference on mining software repositories (MSR 2024)*, 2024. . <https://doi.org/10.1145/3643991.3645081>
- [7] Urvi Awasthi, Lisa Kraye, and Daniel Sack. 2026. TurnStyle: a framework for analyzing human conversational behaviors to predict success in LLM-assisted tasks. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3790459>
- [8] Leonardo Banh, Florian Holldack, and Gero Strobel. 2025. Copiloting the future: How generative AI transforms Software Engineering. *Information and Software Technology* 183, (July 2025), 107751. <https://doi.org/10.1016/j.infsof.2025.107751>
- [9] Jessica Y Bo, Majeed Kazemitabaar, Mengqing Deng, Michael Inzlicht, and Ashton Anderson. 2025. Invisible saboteurs: Sycophantic llms mislead novices in problem-solving tasks. In *ACM conference*, 2025. . <https://doi.org/10.1145/3772318.3791365>
- [10] Jialun Cao, Meiziniu Li, Ming Wen, and Shing-Chi Cheung. 2025. A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair. *Automated Software Engineering* (2025). <https://doi.org/10.1007/s10515-025-00492-x>
- [11] Yu Cheng, Jieshan Chen, Qing Huang, Zhenchang Xing, Xiwei Xu, and Qinghua Lu. 2024. Prompt sapper: a LLM-empowered production tool for building AI chains. *ACM Transactions on Software Engineering and Methodology* (2024). <https://doi.org/10.1145/3638247>
- [12] D. S. Cruzes and T. Dyba. 2011. Recommended Steps for Thematic Synthesis in Software Engineering. In *2011 International Symposium on Empirical Software Engineering and Measurement*, September 2011. IEEE, Banff, AB, 275–284. <https://doi.org/10.1109/ESEM.2011.36>
- [13] Daniela S. Cruzes, Tore Dybå, Per Runeson, and Martin Höst. 2015. Case studies synthesis: a thematic, cross-case, and narrative synthesis worked example. *Empir Software Eng* 20, 6 (December 2015), 1634–1665. <https://doi.org/10.1007/s10664-014-9326-8>
- [14] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming (Jack) Jiang. 2023. GitHub copilot AI pair programmer: Asset or liability? *Journal of Software and System* (2023). <https://doi.org/10.1016/j.jss.2023.111734>
- [15] Gerand Boy Elinzano, Michelle Ching, and Emmanuel Iluzada. 2024. Towards an AI-assisted software developer: The perspective on use of ChatGPT for programming in the philippines. In *Asian CHI symposium 2024*, 2024. . <https://doi.org/10.1145/3704611.3704621>
- [16] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang (Eric) Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent AI systems. In *CHI conference on human factors in computing systems*, 2025. . <https://doi.org/10.1145/3706598.3713581>
- [17] Leonardo Martín Esnaola, Hugo Dionisio Ramón, and Laura Cristina Lanzarini. 2026. Can generative AI bridge the gap? A quasi-experimental study of non-programmers with AI vs. programmers without AI. *Empirical Software Engineering* (2026). <https://doi.org/10.1007/s10664-026-10813-7>
- [18] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-based test-driven interactive code generation: User study and empirical evaluation. *arXiv* (2024). <https://doi.org/10.1109/tse.2024.3428972>
- [19] James Flamino, Mohammed Shahid Modi, Boleslaw K. Szymanski, Brendan Cross, and Colton Mikolajczyk. 2025. Testing the limits of large language models in debating humans. *Scientific Reports* (2025). <https://doi.org/10.1038/s41598-025-98378-1>

- [20] Claudia Flores-Saviaga, Benjamin V. Hanrahan, Kashif Imteyaz, Steven Clarke, and Saiph Savage*. 2025. The impact of generative AI coding assistants on developers who are visually impaired. In *CHI conference on human factors in computing systems (CHI '25)*, 2025. . <https://doi.org/10.1145/3706598.3714008>
- [21] Nat Friedman. 2021. Introducing GitHub Copilot: your AI pair programmer. *The GitHub Blog*. Retrieved April 7, 2026 from <https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/>
- [22] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, and Trung Le. 2023. ChatGPT for vulnerability detection, classification, and repair: How far are we? In *APSEC*, 2023. . <https://doi.org/10.1109/apsec60848.2023.00085>
- [23] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3712005>
- [24] Vahid Garousi, Michael Felderer, and Mika V. Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* 106, (February 2019), 101–121. <https://doi.org/10.1016/j.infsof.2018.09.006>
- [25] Leon Hannig, Annika Bush, Meltem Aksoy, Tim Trappen, Steffen Becker, and Greta Ontrup. 2026. Campus AI vs. commercial AI: Comparing how students and employees perceive their university's LLM chatbot vs. ChatGPT. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3790622>
- [26] Chieh-Yang Huang, Sanjana Gautam, Shannon McClellan Brooks, Ya-Fang Lin, Tiffany Kneare, and Ting-Hao Kenneth Huang. 2024. Inspo: Writing with crowds alongside AI. In *arXiv*, 2024. . <https://doi.org/10.1145/3706599.3720193>
- [27] Qing Huang, Dianshu Liao, Zhenchang Xing, Zhiqiang Yuan, Qinghua Lu, Xiwei Xu, and Jiaxing Lu. 2022. SE factual knowledge in frozen giant code model: a study on FQN and its retrieval. *IEEE Transactions on Knowledge and Data Engineering* (2022). <https://doi.org/10.1109/tkde.2024.3436883>
- [28] Victor Vadmand Jensen, Adam Alami, Anders Rysholt Bruun, and John Stouby Persson. 2025. Managing expectations towards AI tools for software development: a multiple-case study. *Information Systems and e-Business Management* (2025). <https://doi.org/10.1007/s10257-025-00704-7>
- [29] Jie Jiang, Haining Xie, Siqi Shen, Yu Shen, Zihan Zhang, Meng Lei, Yifeng Zheng, Yang Li, Chunyou Li, Danqing Huang, Yinjun Wu, Wentao Zhang, Bin Cui, and Peng Chen. 2025. SiriusBI: a comprehensive LLM-powered solution for data analytics in business intelligence. *Proceedings of the VLDB Endowment* (2025). <https://doi.org/10.14778/3750601.3750610>
- [30] Kailun Jin, Chung-Yu Wang, Hung Viet Pham, and Hadi Hemmati. 2024. Can ChatGPT support developers? An empirical evaluation of large language models for code generation. In *21st international conference on mining software repositories (MSR '24)*, 2024. . <https://doi.org/10.1145/3643991.3645074>
- [31] Bangyan Ju, Jin Yang, Tingting Yu, Tamerlan Abdullayev, Yuanyuan Wu, Dingbang Wang, and Yu Zhao. 2024. A study of using multimodal llms for non-crash functional bug detection in android apps. In *Apsec 2024*, 2024. . <https://doi.org/10.1109/apsec65559.2024.00017>
- [32] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2025. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-024-10594-x>
- [33] Barbara A. Kitchenham and Shari L. Pfleeger. 2008. Personal Opinion Surveys. In *Guide to Advanced Empirical Software Engineering*, Forrest Shull, Janice Singer and Dag I. K. Sjøberg (eds.). Springer, London, 63–92. https://doi.org/10.1007/978-1-84800-044-5_3
- [34] Barbara Kitchenham, Riallette Pretorius, David Budgen, O. Pearl Brereton, Mark Turner, Mahmood Niazi, and Stephen Linkman. 2010. Systematic literature reviews in software engineering – A tertiary study. *Information and Software Technology* 52, 8 (August 2010), 792–805. <https://doi.org/10.1016/j.infsof.2010.03.006>
- [35] Kei Koyanagi, Dong Wang, Kotaro Noguchi, Masanari Kondo, Alexander Serebrenik, Yasutaka Kamei, and Naoyasu Ubayashi. 2024. Exploring the effect of multiple natural languages on code suggestion using GitHub copilot. In *21st international conference on mining software repositories*, 2024. . <https://doi.org/10.1145/3643991.3644917>
- [36] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Why AI agents still need you: Findings from developer-agent collaborations in the wild. <https://doi.org/10.48550/arXiv.2506.12347>
- [37] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild. <https://doi.org/10.48550/arXiv.2506.12347>
- [38] Tzu-Sheng Kuo, Sophia Liu, Quan Ze Chen, Joseph Seering, Amy X. Zhang, Haiyi Zhu, and Kenneth Holstein. 2026. Botender: Supporting communities in collaboratively designing AI agents through case-based provocations. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3790500>
- [39] Thanh Le-Cong, Thanh-Dat Nguyen, Bach Le, and Toby Murray. 2023. Towards reliable evaluation of neural program repair with natural robustness testing. *ACM* (2023). <https://doi.org/10.1145/3716167>
- [40] Yvonna S. Lincoln and Egon G. Guba. 1985. *Naturalistic inquiry*. Sage Publications, Beverly Hills, CA.
- [41] Chunhua Liu, Hong Yi Lin, and Patanamon Thongtanunam. 2025. Too noisy to learn: Enhancing data quality for code review comment generation. In *MSR*, 2025. . <https://doi.org/10.1109/msr66628.2025.00043>

- [42] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Trans. Softw. Eng. Methodol.* (March 2026), 3796507. <https://doi.org/10.1145/3796507>
- [43] Michael Xieyang Liu, Frederick Liu, Alexander J. Fiannaca, Terry Koo, Lucas Dixon, Michael Terry, and Carrie J. Cai. 2024. “We need structured output”: Towards user-centered constraints on large language model output. In *Chi ea '24*, 2024. . <https://doi.org/10.1145/3613905.3650756>
- [44] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. 2024. No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. *IEEE Transactions on Software Engineering* (2024). <https://doi.org/10.1109/tse.2024.3392499>
- [45] Mauricio Monteiro, Bruno Castelo Branco, Samuel Silvestre, Guilherme Avelino, and Marco Tulio Valente. 2025. NoCodeGPT: a no-code interface for building web apps with language models. *arXiv* (2025). <https://doi.org/10.1002/spe.3432>
- [46] Michel Nass, Emil Alégroth, and Robert Feldt. 2023. Improving web element localization by using a large language model. *Software Testing, Verification and Reliability* (2023). <https://doi.org/10.1002/stvr.1893>
- [47] Abdullah Bin Nofal, Hassan Ali, Muhammad Hadi, Aizaz Ahmad, Adnan Qayyum, Aditya Johri, Ala Al-Fuqaha, and Junaid Qadir. 2025. AI-enhanced interview simulation in the metaverse: Transforming professional skills training through VR and generative conversational AI. *Computers and Education: Artificial Intelligence* (2025). <https://doi.org/10.1016/j.caeai.2024.100347>
- [48] Nesreen Otoum and Nuha Elkhaili. 2026. Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review. *IEEE Access* 14, (2026), 7443–7465. <https://doi.org/10.1109/ACCESS.2026.3652325>
- [49] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3697010>
- [50] Eunil Park. 2025. Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10730-1>
- [51] Pat Pataranutaporn, Eunhae Lee, Judith Amores, and Pattie Maes. 2026. Personal validation effect in llms: Positive AI responses bias perceptions of validity, reliability, personalization, and usefulness of fictitious predictions. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3791851>
- [52] Kai Petersen, Robert Feldt, Shahid Mujtaba, and Michael Mattsson. 2008. Systematic mapping studies in software engineering. In *Proceedings of the 12th international conference on Evaluation and Assessment in Software Engineering*. 68–77. <https://doi.org/10.5555/2227115.2227123>
- [53] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology* 64, (August 2015), 1–18. <https://doi.org/10.1016/j.infsof.2015.03.007>
- [54] Matei-Dan Popovici. 2024. ChatGPT in the classroom. Exploring its potential and limitations in a Functional Programming course. *arXiv* (2024). <https://doi.org/10.1080/10447318.2023.2269006>
- [55] Piotr Przyms, Mikołaj Fejzer, Jakub Narębski, and Krzysztof Stencel. 2024. How I learned to stop worrying and love ChatGPT. In *21st international conference on mining software repositories (MSR '24)*, 2024. . <https://doi.org/10.1145/3643991.3645073>
- [56] Rachna Raj and Diego Elias Costa. 2024. The role of library versions in Developer-ChatGPT conversations. In *Proceedings of MSR '24: Proceedings of the 21st international conference on mining software repositories (MSR 2024)*, 2024. . <https://doi.org/10.1145/3643991.3645075>
- [57] Charvi Rastogi, Marco Tulio Ribeiro, Nicholas King, Harsha Nori, and Saleema Amershi. 2023. Supporting human-AI collaboration in auditing llms with llms. In *AAAI/ACM conference on AI, ethics, and society (AIES '23)*, 2023. . <https://doi.org/10.1145/3600211.3604712>
- [58] Kevin Roitero, Dustin Wright, Michael Soprano, Isabelle Augenstein, and Stefano Mizzaro. 2025. Efficiency and effectiveness of LLM-based summarization of evidence in crowdsourced fact-checking. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval*, 2025. . <https://doi.org/10.1145/3726302.3729960>
- [59] Amirali Sajadi, Binh Le, Anh Nguyen, Kostadin Damevski, and Preetha Chatterjee. 2025. Do LLMs consider security? an empirical study on responses to programming questions. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10658-6>
- [60] Raphael Serafini, Asli Yardim, and Alena Naiakshina. 2025. Exploring the impact of intervention methods on developers’ security behavior in a manipulated ChatGPT study. In *CHI conference on human factors in computing systems (CHI '25)*, 2025. . <https://doi.org/10.1145/3706598.3713989>
- [61] Leuson Da Silva, Jordan Samhi, and Foutse Khomh. 2025. LLMs and stack overflow discussions: Reliability, impact, and challenges. *Journal of Systems and Software* (2025). <https://doi.org/10.1016/j.jss.2025.112541>
- [62] Vedant Das Swain, Qiuyue “Joy” Zhong, Jash Rajesh Parekh, Yechan Jeon, Roy Zimmermann, Mary P Czerwinski, Jina Suh, Varun Mishra, Koustuv Saha, and Javier Hernandez. 2025. AI on my shoulder: Supporting emotional labor in front-office roles with an LLM-based empathetic coworker. In *CHI conference on human factors in computing systems (CHI '25)*, 2025. . <https://doi.org/10.1145/3706598.3713705>
- [63] Florian Tambon, Arghavan Moradi-Dakheel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2025. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10614-4>

- [64] Jiesie Tie, Bingsheng Yao, Tianshi Li, Hongbo Fang, Syed Ishtiaque Ahmed, Dakuo Wang, and Shurui Zhou. 2026. “Should I give up now?” investigating LLM pitfalls in software engineering. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3801972>
- [65] Utku Boran Torun, Mehmet Taha Demircan, Mahmut Furkan Gön, and Eray Tüzün. 2026. Past, present, and future of bug tracking in the generative AI era. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3806655>
- [66] Christoph Treude and Marco A. Gerosa. 2025. How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering. <https://doi.org/10.48550/arXiv.2501.08774>
- [67] Mario Truss. 2026. PersonaCite: VoC-grounded interviewable agentic synthetic AI personas for verifiable user and design research. In *Extended abstracts of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772363.3798543>
- [68] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabic, Massimiliano Di Penta, and Gabriele Bavota. 2024. Unveiling ChatGPT’s usage in open source projects: a mining-based study. In *21st international conference on mining software repositories (MSR ’24)*, 2024. . <https://doi.org/10.1145/3643991.3644918>
- [69] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI conference on human factors in computing systems extended abstracts*, 2022. . <https://doi.org/10.1145/3491101.3519665>
- [70] Eszter Vigh, Freja Sunesen, and Louise Barkhuus. 2026. “AI does not understand the real world.”: AI Augmented Software Development. In *Extended abstracts of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772363.3799079>
- [71] Huaning Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. 2025. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities. <https://doi.org/10.48550/arXiv.2508.11126>
- [72] Xingbo Wang, Janessa Griffith, Daniel A. Adler, Joey Castillo, Tanzeem Choudhury, and Fei Wang. 2025. Exploring personalized health support through data-driven, theory-guided llms: a case study in sleep health. In *CHI conference on human factors in computing systems (CHI ’25)*, 2025. . <https://doi.org/10.1145/3706598.3713852>
- [73] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: survey, landscape, and vision. *Automated Software Engineering* 32, 2 (August 2025). <https://doi.org/10.1007/s10515-025-00544-2>
- [74] Miku Watanabe, Yutaro Kashiwa, Bin Lin, Toshiki Hirao, Ken’ichi Yamaguchi, and Hajimu Iida. 2024. On the use of ChatGPT for code review: Do developers like reviews by ChatGPT? In *EASE 2024: 28th international conference on evaluation and assessment in software engineering*, 2024. . <https://doi.org/10.1145/3661167.3661183>
- [75] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2026. On the use of agentic coding: An empirical study of pull requests on GitHub. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3798166>
- [76] Justin D. Weisz, Shraddha Vijay Kumar, Michael Muller, Karen-Ellen Browne, Arielle Goldberg, Katrin Ellice Heintze, and Shagun Bajpai. 2025. Examining the use and impact of an AI code assistant on developer productivity and experience in the enterprise. In *Extended abstracts of the CHI conference on human factors in computing systems (CHI EA ’25)*, 2025. . <https://doi.org/10.1145/3706599.3706670>
- [77] Cheng Wen, Yuandao Cai, Bin Zhang, Jie Su, Zhiwu Xu, Dugang Liu, Shengchao Qin, Zhong Ming, and Tian Cong. 2024. Automatically inspecting thousands of static bug warnings with large language model: How far are we? *ACM Trans. Knowl. Discov. Data.* (2024). <https://doi.org/10.1145/3653718>
- [78] Ratnadira Widyasari, Ting Zhang, Abir Bouraffa, Walid Maalej, and David Lo. 2024. Explaining explanations: An empirical study of explanations in code reviews. *ACM* (2024). <https://doi.org/10.1145/3708518>
- [79] Roel Wieringa, Neil Maiden, Nancy Mead, and Colette Rolland. 2006. Requirements engineering paper classification and evaluation criteria: a proposal and a discussion. *Requirements Eng* 11, 1 (March 2006), 102–107. <https://doi.org/10.1007/s00766-005-0021-6>
- [80] Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *CHI conference on human factors in computing systems*, 2022. . <https://doi.org/10.1145/3491102.3517582>
- [81] Zhipeng Xue, Xiaoting Zhang, Zhipeng Gao, Xing Hu, Shan Gao, Xin Xia, and Shanping Li. 2026. Clean code, better models: Enhancing LLM performance with smell-cleaned dataset. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3793252>
- [82] Xiao Yu, Lei Liu, Xing Hu, Jacky Wai Keung, Jin Liu, and Xin Xia. 2024. Fight fire with fire: How much can we trust ChatGPT on source code-related tasks? *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* (2024). <https://doi.org/10.1109/tse.2024.3492204>
- [83] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2023. AppAgent: Multimodal agents as smartphone users. In *arXiv*, 2023. . <https://doi.org/10.1145/3706598.3713600>
- [84] Yuxia Zhang, Zhiqing Qiu, Klaas-Jan Stol, Wenhui Zhu, Jiaxin Zhu, Yingchen Tian, and Hui Liu. 2024. Automatic commit message generation: a critical review and directions for future work. *IEEE Transactions on Software Engineering* (2024). <https://doi.org/10.1109/tse.2024.3364675>
- [85] Xiyu Zhou, Peng Liang, Beiqi Zhang, Zengyang Li, Aakash Ahmad, Mojtaba Shahin, and Muhammad Waseem. 2024. Exploring the problems, their causes and solutions of AI pair programming: a study on GitHub and stack overflow. *Journal of Systems and Software* (2024). <https://doi.org/10.1016/j.jss.2024.112204>

- [86] Terry Yue Zhuo, Junda He, Jiamou Sun, Zhenchang Xing, David Lo, John Grundy, and Xiaoning Du. 2025. Identifying and mitigating API misuse in large language models. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* (2025). <https://doi.org/10.1109/tse.2026.3651566>

APPENDIX A: PROTOCOL ARTEFACTS

A.1 Search Strings

Six search strings were designed for Scopus, targeting title, abstract, and keywords to span the six scope sub-spaces that motivate the study:

- **S1 — AI coding assistants (adoption-framed).** ("AI coding assistant" OR "AI code generation" OR "LLM-based coding" OR "LLM coding assistant") AND ("usage" OR "adoption" OR "practice" OR "experience")
- **S2 — Named tool mentions (concrete surfaces).** ("GitHub Copilot" OR "ChatGPT" OR "Antigravity" OR "Claude Code" OR "Cursor" OR "Windsurf" OR "Devin" OR "Power Platform" OR "n8n" OR "AppSheet" OR "Amazon Q" OR "Codex") AND ("software engineering" OR "software development" OR "application development")
- **S3 — Agentic software engineering (empirical-framed).** ("agentic software engineering" OR "SE agent" OR "coding agent") AND ("empirical" OR "case study" OR "survey" OR "interview")
- **S4 — AI-assisted programming (population-framed).** ("AI-assisted programming" OR "AI pair programming" OR "AI-assisted development") AND ("developer" OR "software engineer" OR "SWE" OR "practitioner" OR "student" OR "citizen developer" OR "business user")
- **S5 — LLMs × SDLC activities.** ("LLM" OR "large language model") AND ("code review" OR "testing" OR "debugging" OR "deployment") AND ("human" OR "user")
- **S6 — Low-code / no-code / citizen-developer AI.** ("low-code" OR "no-code" OR "citizen developer" OR "workflow automation") AND ("AI" OR "LLM" OR "large language model" OR "copilot" OR "agent") AND ("software development" OR "software engineering" OR "application development" OR "app development")

A parallel ACM Digital Library run used ACM-DL syntax for the same six sub-spaces. Snowballing seeds: Wang et al. [71], Wang et al. [73], Otoum & Elkhaili [48], Ajimati et al. [1], and Liu et al. [42].

A.2 Inclusion and Exclusion Criteria

The protocol declares five inclusion criteria and six exclusion criteria. The authoritative source is artifacts/protocol/inclusion_exclusion.md Criteria are enforced across three layers: **Q** = Scopus query (Layer 1 — mechanical filter), **S** = manual title / abstract screening (Layer 2), **P** = post-retrieval script (Layer 3).

Table 4: Inclusion criteria (IC1–IC5)

Code	Rule	Layer
IC1	Publication between 1 January 2022 and the search cut-off (2026-04-13).	Q
IC2	Reports at least one empirical study (survey, interview, case study, experiment, field study, mining study) of human interaction with SE agents, AI coding tools, or AI-enhanced low-code / no-code and workflow-automation platforms used for software or application development.	S
IC3	Peer-reviewed journal article or conference paper (Scopus SRCTYPE journal or conference proceedings; DOCTYPE article or conference paper).	Q
IC4	Written in English.	Q
IC5	At least 4 pages in length.	P

Table 5: Exclusion criteria (EC1–EC6)

Code	Rule	Layer
EC1	The paper only proposes or builds an agent system without reporting usage evidence. Classified as Solution Proposal per Wieringa et al. 2006 and excluded because it contributes to the supply side, not the demand side this SMS maps.	S
EC2	The paper is a benchmark study evaluating model or agent capabilities without human users. Classified as Validation Research per Wieringa et al. 2006 and excluded because it lacks the empirical human-interaction evidence required by IC2.	S
EC3	The paper is outside the software engineering domain. This is a Layer-2 manual screening check; SUBJAREA(COMP) is deliberately not applied at Layer 1 to avoid silently dropping citizen-developer and cross-disciplinary venues.	S
EC4	The paper is a secondary study (existing survey or systematic literature review). DOCTYPE(re) is excluded at Layer 1; manual confirmation at Layer 2. Secondary studies are used as snowball seeds only.	Q + S
EC5	The paper is a short paper (< 4 pages), poster, or extended abstract. Partially filtered at Layer 1 by DOCTYPE exclusions (ed, no, le, sh, er, rt); confirmed by the page-count script at Layer 3.	Q + P
EC6	The full text is not retrievable within two weeks of identification. Logged in <code>decision_register.csv</code> with the retrieval attempts and deadline.	P

A.3 Per-criterion exclusion counts (execution-time tally)

The Phase-2 screening engine recorded exclusion rulings under an execution-time labelling convention that mirrored the inclusion criteria: “fails IC_i” was logged as “EC_i” in the PRISMA bookkeeping for five of the six criteria. The tally below preserves those execution-time labels for PRISMA transparency; the right-hand column maps each legacy label to the protocol-defined exclusion in Table 5.

Table 6: Per-criterion exclusion-ruling tally

Execution-time label	Rulings	Share of exclusions	Maps to protocol-defined rule
Legacy EC1 (out-of-window)	1,671	39.9%	Failure of IC1 (publication window)
Legacy EC2 (non-empirical)	588	14.0%	EC1 (Solution Proposal) combined with EC2 (Validation Research); a future re-tally will split this total by Wieringa class
Legacy EC3 (out-of-scope)	1,200	28.6%	EC3 (outside software engineering domain)
Legacy EC4 (non-English)	668	15.9%	Failure of IC4 (English-language requirement)
Legacy EC5 (short-paper, post-IC5)	3	0.1%	EC5 (short paper / poster)
Legacy EC6 (non-primary-study)	58	1.4%	EC4 (secondary study)
Legacy IC1 (restored at Layer 2 after Layer-1 false rejection)	2	0.0%	Layer-1 filter overturned at Layer 2
Total rulings	4,190	100.0%	—

A.4 Modality -- F1–F5 codebook

The five facets are pre-committed at Phase 1. Each facet value has an operational definition, positive and negative exemplars, and a borderline rule.

F1 — Contribution type (Wieringa et al. 2006 six-class vocabulary).

Table 7: F1 - Contribution type

Value	Decision rule
Evaluation Research	The paper investigates an artefact <i>in context</i> (industrial, classroom, field) and evaluates its fit against that context. Default for empirical papers that study a deployed tool.

Value	Decision rule
Validation Research	The paper evaluates an artefact under constructed / controlled conditions (lab experiment, benchmark). Default for controlled experiments and method evaluations.
Solution Proposal	The paper proposes a novel technique with a small proof-of-concept evaluation; the main contribution is the proposal, not the evaluation.
Personal Experience	The paper reports a single practitioner’s or team’s reflective experience without systematic evaluation.
Philosophical	The paper proposes a new conceptual lens.
Opinion	The paper offers a positional argument.

F2 — Research methodology. Seven canonical values: Experiment, Case Study, Mixed, Survey, Mining Study, Interview, Field Study. Mixed is used when two or more primary methods contribute comparable evidentiary weight; the component methods are recorded in the extraction-form notes column.

F3 — Population × Context. Population axis: Professional SWE, Student, Citizen Developer, OSS Contributor, Mixed. Context axis: Industry, Education, OSS, Lab, Mixed. F3 is set to NaN where a paper carries no study-population description.

F4 — SDLC activity. Nine values: Requirements, Design, Coding, Testing, Code Review, Debugging, CI/CD, Documentation, and Project Management. F4 is multi-valued in storage (pipe-separated); a paper describing a tool used for both coding and debugging is legitimate evidence for both activities. Counting-unit disclosures accompany every F4-derived table or figure (distinct papers vs. paper–activity bindings).

F5 — Agent / tool profile. Composite tag with two sub-facets. *Development paradigm* (single-valued): Pro-code, Low-code, No-code. *Interaction modality* (multi-select): Autocomplete, Conversational, IDE-Integrated, Autonomous. Multi-select modality is essential: several mainstream tools span Autocomplete, Conversational, and IDE-Integrated simultaneously.

A.5. Extraction-form schema

Each included paper populates a row of artifacts/extraction/extraction_matrix.csv with bibliographic metadata (paper_id, title, authors, venue, year, DOI), F1–F5 values, sample characterization (participants, context, tool, task), raw verbatim passages describing human–AI interaction (stored separately under artifacts/extraction/raw_passages/), per-paper capability annotations against the 19-capability vocabulary (§A.6), and Mode-A/Mode-B status. Full-text retrieval used a four-tier cascade: Unpaywall → Semantic Scholar → arXiv search → manual acquisition.

A.6 Capability-list harmonization

Nineteen unified capability IDs were built by overlaying the capability vocabularies of the three reference surveys [48, 71, 73] and collapsing near-synonyms. Each capability carries an in_source_survey count in {1, 2, 3} that feeds the RQ3 priority formula.

Table 8: Harmonized capability vocabulary with reference-survey prevalence

Capability ID	Label	In source surveys
CAP_CODEGEN	Code Generation	3
CAP_CODECOMP	Code Completion	3
CAP_PROGREPAIR	Program Repair / Bug Fixing	3
CAP_CODETRANS	Code Translation	3
CAP_CODESUM	Code Summarization	1
CAP_TESTING	Test Generation and Execution	3
CAP_DEBUGGING	Debugging and Fault Localization	3
CAP_CODEREVIEW	Automated Code Review	3
CAP_REFACTORING	Code Refactoring	2
CAP_VULNDET	Vulnerability Detection	2

Capability ID	Label	In source surveys
CAP_DOCGEN	Documentation Generation	3
CAP_REQENG	Requirements Engineering	2
CAP_SYSDESIGN	System Design and Architecture	1
CAP_COMMITMSG	Commit Message Generation	1
CAP_CODESEARCH	Code Search and Retrieval	1
CAP_PLANNING	Task Planning and Decomposition	2
CAP_MULTIAGENT	Multi-Agent Collaboration	2
CAP_CICD	CI/CD and Deployment Automation	2
CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	3

APPENDIX B: RQ1–RQ2 ARTEFACTS

B.1. Capability, modality, and interaction mode - three-way disambiguation

These three terms are distinct and are kept distinct throughout the study to avoid the category error that the prior literature sometimes commits.

- **Capability** (§A.6) is a supply-side tool attribute: what the AI or agent can do (for example, generate code, repair bugs, translate between languages). Each primary study annotates one or more capabilities; the 19-capability vocabulary is harmonized from the three reference surveys.
- **Modality** (§A.4, F5 sub-facet) is a supply-side tool attribute: how the tool surfaces AI capability to the user (Autocomplete, Conversational, IDE-Integrated, Autonomous). F5 modality is multi-select because mainstream tools span multiple modalities simultaneously.
- **Interaction mode** (§B.3) is an inductively derived demand-side human–AI pattern: how the human engages with the AI along a delegation-depth axis (Inline Completion, Conversational Prompting, Visual / Declarative Composition, Review & Validation, Delegated Task Execution). Mode is derived from open coding of passages describing human–AI interaction; it is not a pre-committed facet.

To clarify these conceptual boundaries: capability defines the functional task (what the AI can do), while modality defines the interface mechanism (how that capability is surfaced). In contrast, the distinction between modality and interaction mode separates an intrinsic system attribute from an observed human behavioral pattern. Because these dimensions are orthogonal, a single system modality can support multiple user interaction modes. For instance, a tool featuring a conversational modality can dynamically facilitate both Mode 2 (Conversational Prompting) and Mode 4 (Review & Validation) engagements, depending entirely on whether the practitioner is actively prompting for novel output or critically evaluating a generated response.

B.2. Per-figure image-explanation notes (Figures 2 to 5)

These accompany the main paper’s Figures 2 to 5 with counting-unit disclosures and interpretation caveats.

- **Figure 2 (F2 research-methodology distribution).** This figure quantifies the distinct paper count across empirical research methods ($N = 640$). The Mixed category (139 papers; 21.7%) aggregates studies that explicitly combine two or more canonical methods with comparable evidentiary weight. To preserve analytical granularity, the component methods of these mixed studies are maintained discretely within the underlying dataset and can be systematically decomposed upon request.
- **Figure 3 (Year $\times$ SDLC activity $\times$ developer population bubble plot).** This visualization maps the distinct paper count for each year-by-SDLC intersection, faceted by F3 target populations (bubble area denotes volume). The effective sample size for this figure is reduced to 392 (from 640) because 246 papers lack an explicit human-participant description, predominantly algorithmic benchmark evaluations, and are

thus excluded. Because SDLC activity is a multi-valued attribute, a single paper addressing multiple phases (e.g., Coding and Debugging) independently increments the count in all applicable cells. Given the pronounced sparsity in the Citizen Developer (19 papers) and OSS Contributor (3 papers) facets, interpretations must rely on aggregate trends across the full visualization rather than drawing inferences from these isolated panels.

- **Figure 4 (SDLC × year evidence heatmap).** This heatmap illustrates the temporal evolution of research focus across the software lifecycle. Each cell quantifies the distinct paper count for a specific SDLC phase within a given year. The visualization clearly highlights a persistent dominance of code-centric activities (Coding, Testing, and Debugging) throughout the observation window, juxtaposed against a chronic empirical scarcity in upstream planning (Requirements, Project Management) and downstream operational (CI/CD) activities.
- **Figure 5 (F5 tool profile).** This stacked bar chart maps tool interaction modalities (Autocomplete, Conversational, IDE-Integrated, Autonomous) against development paradigms (Pro-code, Low-code, No-code). The paradigm axis is mutually exclusive and single-valued (figure-plotted distinct-paper counts: Pro-code 537, Low-code 30, No-code 72; total n=639 of 640 corpus papers, the single excluded paper having a NaN modality and therefore no plottable stack segment). While the base paradigm totals are mutually exclusive and single-valued (Pro-code: 538; Low-code: 30; No-code: 72), the internal modality classifications reflect multi-select attributes, acknowledging that modern tools frequently exhibit hybrid interfaces. Consequently, the cumulative count of interaction modalities within a given paradigm stack may exceed the absolute paper count for that paradigm.

B.3. Per-mode operational definitions and paraphrased exemplars

Each mode carries an operational definition, IS / IS-NOT distinguishing criteria, and at least two paraphrased exemplars traced to source papers. Every exemplar is paraphrase-linter clean against the raw passage.

Mode 1: Inline Completion

- **Operational Definition:** The human writes code in-flow while the AI proposes short continuations (ranging from single tokens to a few lines) that the human immediately accepts, modifies, or rejects. The fundamental unit of interaction is the single accept/reject decision; no turn-based dialogue is required.
- **Boundary Criteria:** Includes interactions where the continuation appears inline within the editor, decisions are executed without a separate conversational turn, and the AI takes no multi-step initiative. Excludes iterative prompt refinement (Mode 2) and any instance where the AI executes a task without an immediate token-level decision (Mode 5).
- **Empirical Exemplars:** (1) Vaithilingam et al.’s [69] controlled study of 24 programmers compared an inline Copilot surface—proposing per-line continuations—against a chat-style GPT-3 workflow; the inline accept/reject surface typifies Mode 1. (2) Developers routinely invoking autocomplete to finish the current line during active typing, with acceptance or rejection occurring without opening a chat turn.

Mode 2: Conversational Prompting

- **Operational Definition:** The human articulates intent via natural language, and the AI generates a multi-line response. The human reads, evaluates, and iterates over multiple turns until the output is usable. The fundamental unit of interaction is the turn. Delegation scope is bounded by a single prompt–response cycle, with multiple cycles compounding into a session.
- **Boundary Criteria:** Includes turn-based exchanges via chat or comparable interfaces where the human re-prompts to steer output, and the scope of any single AI action is strictly bounded by the most recent

prompt. Excludes inline accept/reject actions (Mode 1), visual/declarative inputs (Mode 3), and multi-step execution without intermediate steering (Mode 5).

- **Empirical Exemplars:** (1) Participants reporting a dominant strategy of authoring natural-language prompts—often embedded as in-file comments—to articulate desired code behavior, refining across turns until the output matched intent. (2) A debugging turn involving pasting a code fragment into a chat interface, appending the observed exception, and requesting a diagnosis and fix; the dialogue extends across subsequent turns if the initial response is incomplete.

Mode 3: Visual / Declarative Composition

- **Operational Definition:** The human composes software by arranging visual, structural, or declarative elements (e.g., drag-and-drop blocks, flowchart nodes, configuration forms, or natural-language-to-workflow pipelines). The AI subsequently generates the underlying implementation (logic, data models, workflow steps) from this declarative specification. The fundamental unit of interaction is the declarative specification itself, not a free-text prompt.
- **Boundary Criteria:** Includes visual or declarative surfaces where the human's input is structural rather than conversational; encompasses Low-code, No-code, and visual-AI-platform evidence. Excludes free-text prompt-and-response interfaces (Mode 2) and code-editor inline completions (Mode 1).
- **Empirical Exemplars:** (1) A prompt-composition tool that surfaces construction through a visual analytics interface, allowing the user to manipulate structural elements (exploration panels, perturbation widgets, test harnesses) rather than authoring chat turns. (2) Structural diagrams (visualized code trees and logic flows) enabling practitioners to compose logic declaratively; the diagram itself acts as the surface of interaction.

Mode 4: Review & Validation

- **Operational Definition:** The AI or the human performs an evaluative reading of code or AI output—detecting hallucinations, assessing quality, localizing defects, or providing/soliciting feedback. The fundamental unit of interaction is the evaluative judgment. The delegation axis interprets this mode bidirectionally as either "AI-as-subject-of-review" or "AI-as-review-provider."
- **Boundary Criteria:** Includes the explicit evaluation of code or AI output for correctness, quality, or trustworthiness, applying to both human-reviewing-AI and AI-reviewing-human directions. Excludes the generation of new code (Modes 1, 2, and 5) and the refinement of a prompt where evaluation remains internal to a conversational turn (Mode 2).
- **Empirical Exemplars:** (1) Evaluators sorting AI-generated unit tests into distinct defect families (unresolved symbols, parameter mismatches, abstract-class instantiations) traceable to hallucinations; the resulting work product is purely evaluative. (2) Reviewers applying an explicit four-axis rubric to AI-produced code (artifact quality, correctness, turnaround speed, and human-baseline comparison), where the interaction unit is the judgment rather than the generation step.

Mode 5: Delegated Task Execution

- **Operational Definition:** The human scopes a broad software engineering task (e.g., generating a test suite, fixing a bug, refactoring a module, or building a workflow), and the AI executes it end-to-end or across multiple sub-steps. The human reviews the output at designated checkpoints or upon completion. The fundamental unit of interaction is the overarching task assignment. (Note: This mode consolidates the provisional HITL Delegation and Autonomous Orchestration categories from the initial protocol).

- **Boundary Criteria:** Includes instances where the AI takes multi-step initiative (e.g., orchestrating an agentic workflow or walking a debugging session), and the human steers primarily at task boundaries rather than per-token. Excludes single-turn prompt-responses (Mode 2) and token-level suggestions (Mode 1).
- **Empirical Exemplars:** (1) An agentic framework decomposing a user-supplied test-generation task into sub-steps, authoring the tests, executing them against the target, and reporting a consolidated result; human intervention occurs strictly at the checkpoint. (2) Weisz et al. [76] reporting a multi-week industrial deployment where a code-assistant agent assumed multi-step tasks (generate, refactor, document) under human checkpoint review; the interaction unit was the holistic assignment.

Residuals (Unclassified Evidence)

- **Operational Definition:** Canonical labels that describe empirical evidence about human–AI interaction rather than documenting a mode-defining interaction episode. This encompasses outcomes of use (192 labels), constraints and barriers (119 labels), meta/reflective commentary (101 labels), and tool affordances (56 labels).
- **Note:** The Residuals layer forms a coherent and distinct analytical sub-structure and is reported jointly alongside the five-mode partition.

B.4 Mode × cross-tabs (summary)

- **Interaction Modes Across the SDLC.** An analysis of row percentages (artifacts/analysis/rq2_variation/mode_x_sdlic.md) reveals that the Coding phase universally dominates across all interaction modes: Mode 1 (80.0%), Mode 2 (59.6%), Mode 3 (81.8%), Mode 4 (52.8%), Mode 5 (57.9%), and Residuals (60.8%). Notably, Mode 5 exhibits the most substantial CI/CD footprint (11 papers; 9.6%), aligning with the delegation of autonomous execution tasks. Conversely, Mode 4 captures the highest share of Code Review activities (25.5%), corroborating its inherently evaluative stance.
- **Interaction Modes and Tool Profiles (F5).** Tool profile distributions (artifacts/analysis/rq2_variation/mode_x_tool_profile.md) are anchored by a denominator of 255, representing the count of distinct papers carrying both an F5 paradigm value and at least one Residuals-classified label, rather than the total count of all passage-bearing papers. Within this subsample, restricted to the Residuals row (255 papers), the Pro-code paradigm dominates at 87.5%. No-code papers (22 of 255) are distributed across Modes 2 (11), 3 (1), 4 (11), and 5 (10). Low-code papers (10 of 255) are similarly split among Modes 2 (3), 3 (2), 4 (3), and 5 (4). Mode 3 (Visual / Declarative Composition) is unique in capturing a substantial Low-code share (18.2% of its 11-paper subsample, compared to $\leq 3.5\%$ across all other modes), which directly reflects its underlying visual and declarative construct. Furthermore, Mode 3's No-code share (9.1%) is comparable to that of Modes 2 (11.7%), 4 (10.4%), and 5 (8.8%), indicating that No-code evidence is dispersed across multiple interaction paradigms rather than isolated to a single mode.
- **Demographics and Contextual Distribution (F3).** Population and context distributions (artifacts/analysis/rq2_variation/mode_x_population_context.md; Residuals-row denominator $N = 255$) indicate a heavy concentration in professional and academic settings. The Professional SWE / Industry intersection accounts for 21.2% (54 papers), followed by Student / Education at 16.9% (43 papers), Mixed / Lab at 10.6% (27 papers), and Professional SWE / Lab at 5.1% (13 papers). In stark contrast, Citizen-Developer and OSS-Contributor populations jointly contribute a mere 2.7% (7 papers). This density remains too thin to support robust, mode-level prevalence claims for these specific cohorts.

APPENDIX C: RQ3 AND TRUSTWORTHINESS ARTEFACTS

C.1 Objectives × reference surveys — novelty-comparison matrix (4 × 3)

Every cell record whether a given research objective is delivered by a given reference survey. The matrix documents zero overlap at any cell; cells are populated by inspecting each survey’s scope statement, research-question set, and findings section:

Table 9: Novelty-comparison matrix

Objective \ Reference survey	Wang et al. [71]	Wang et al. [73]	Otoun & Elkhaili [48]
(i) 2022–2026 empirical landscape along five facets	—	—	—
(ii) 5-mode interaction taxonomy along a delegation-depth axis	—	—	—
(iii) 19 × 9 capability × SDLC gap matrix (supply vs demand)	—	—	—
(iv) Wieringa-typed 7-call research agenda (Calls 1–6 corpus-internal, covering 42 gaps + 12 empty cells; Call 7 cross-paradigm taxonomy extension addressing the Pro-code-dominance transferability boundary)	—	—	—

All twelve cells are empty: no reference survey delivers any of the four objectives. The novelty claim is therefore not a marginal improvement over prior work; it is a demand-side mapping that sits adjacent to the three supply-side surveys.

C.2 Eight transferability boundaries

Drawn from Lincoln & Guba and adapted to SE by Cruzes & Dybå [12]. Each boundary names (a) the scope limit, (b) how it was introduced, (c) which findings it constrains.

1. Time window (2022–2026). Evidence concentration is 2022–2026 only; trends do not extrapolate pre-2022. Limits: RQ1 landscape, RQ2 mode prevalence.
2. Venue set (Scopus + ACM DL + five seed surveys; no grey literature). Any practice reported only in vendor blog posts or industry reports is invisible. Limits: RQ1 coverage.
3. Language (English-only). Region-specific practice published in non-English venues is out of scope. Limits: RQ1 regional coverage.
4. Paradigm skew (Pro-code dominance). 84.1% of F5-profiled papers are Pro-code. Limits: RQ2 Mode 3 prevalence; RQ3 per-paradigm gap inference (Low-code 30 papers, No-code 72 papers).
5. F3 population coverage (61.6% canonical). 246 of 640 papers carry F3 = NaN. Limits: RQ1 population-faceted figures; RQ2 Mode × population cross-tabs.
6. Mode 1 thin evidence (4 of 707 canonical labels). Publication bias, not usage sparsity. Limits: any Mode 1 prevalence claim.
7. Passage-bearing / no-passage coding coverage (290 / 350). The Mode A / Mode B split is 208 / 432; abstract-only papers (Mode B) typically cannot support passage-level qualitative analysis, but 82 of 432 Mode B papers had abstracts rich enough to yield at least one passage and therefore enter the 290-paper RQ2 grounding base. The remaining 350 papers (all Mode B) contribute to RQ1 and RQ3 only. Limits: RQ2 derivation denominator. ERP3 interviews are the remedy.
8. ERP3 hand-off scope. The research agenda is a pre-registered priority set, not a validated finding. Limits: agenda authority; ERP3 may validate, extend, or refute.

C.3 P20 / P25 / P33 / P40 threshold sensitivity

Table 10: RQ3 gap-rule threshold sensitivity

Percentile	Threshold	Gap cells	Empty cells	Normal cells	Δ vs P25
P20	2.000	42	12	117	0
P25 (primary)	2.000	42	12	117	—
P33 (sensitivity)	3.140	53	12	106	+11
P40	5.000	72	12	87	+30

Threshold Sensitivity and Conservative Bias. As detailed in Appendix C.3, the 20th (P20) and 25th (P25) percentiles yield identical threshold values for this corpus (2.000) due to the right-skewed nature of the non-zero cell distribution. The count of entirely empty cells ($N = 12$) naturally remains invariant across all percentile adjustments. Crucially, the gap classification demonstrates monotonic stability: no cell flagged within the primary P25 gap set loses its classification at higher thresholds, establishing a strict subset sequence ($P25 \subseteq P33 \subseteq P40$). Elevating the threshold to P33 introduces 11 additional cells to the gap set. Notably, this expansion captures emerging intersections in agentic capabilities; specifically, CAP_MULTIAGENT applied to Project Management, and CAP_PLANNING applied to CI/CD and Code Review. The omission of these specific gaps under the primary P25 rule highlights a conservative bias inherent in the strict primary threshold.

C.4 Full 42-row prioritized gap table

Ordered by priority (desc), ties broken by capability-ID then SDLC activity.

Table 11: Full 42-row prioritized gap table

Rank	Capability	Label	SDLC	Evidence	Priority	Bucket
1	CAP_CODECOMP	Code Completion	Design	1	1.50	design_gap
2	CAP_CODECOMP	Code Completion	Requirements	1	1.50	design_gap
3	CAP_CODETRANS	Code Translation	CI/CD	1	1.50	organizational_readiness_barrier
4	CAP_CODETRANS	Code Translation	Project Management	1	1.50	organizational_readiness_barrier
5	CAP_CODETRANS	Code Translation	Requirements	1	1.50	design_gap
6	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Code Review	1	1.50	adoption_lag
7	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Documentation	1	1.50	adoption_lag
8	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Project Management	1	1.50	adoption_lag
9	CAP_MULTIAGENT	Multi-Agent Collaboration	CI/CD	1	1.00	adoption_lag
10	CAP_MULTIAGENT	Multi-Agent Collaboration	Code Review	1	1.00	adoption_lag
11	CAP_MULTIAGENT	Multi-Agent Collaboration	Design	1	1.00	design_gap
12	CAP_VULNDET	Vulnerability Detection	CI/CD	1	1.00	organizational_readiness_barrier

Rank	Capability	Label	SDLC	Evidence	Priority	Bucket
13	CAP_VULNDET	Vulnerability Detection	Project Management	1	1.00	organizational_readiness_barrier
14	CAP_CODESEARCH	Code Search & Retrieval	Code Review	1	0.50	organizational_readiness_barrier
15	CAP_CODESEARCH	Code Search & Retrieval	Debugging	1	0.50	organizational_readiness_barrier
16	CAP_CODESEARCH	Code Search & Retrieval	Documentation	1	0.50	organizational_readiness_barrier
17	CAP_CODESEARCH	Code Search & Retrieval	Testing	1	0.50	organizational_readiness_barrier
18	CAP_CODESUM	Code Summarisation	Requirements	1	0.50	design_gap
19	CAP_COMMITMSG	Commit Message Generation	CI/CD	1	0.50	organizational_readiness_barrier
20	CAP_COMMITMSG	Commit Message Generation	Code Review	1	0.50	organizational_readiness_barrier
21	CAP_COMMITMSG	Commit Message Generation	Debugging	1	0.50	organizational_readiness_barrier
22	CAP_COMMITMSG	Commit Message Generation	Project Management	1	0.50	organizational_readiness_barrier
23	CAP_COMMITMSG	Commit Message Generation	Testing	1	0.50	organizational_readiness_barrier
24	CAP_CICD	CI/CD & Deployment Automation	Code Review	2	0.00	organizational_readiness_barrier
25	CAP_CICD	CI/CD & Deployment Automation	Design	2	0.00	design_gap
26	CAP_CICD	CI/CD & Deployment Automation	Documentation	2	0.00	organizational_readiness_barrier
27	CAP_CICD	CI/CD & Deployment Automation	Project Management	2	0.00	organizational_readiness_barrier
28	CAP_CODEREVIEW	Automated Code Review	Project Management	2	0.00	organizational_readiness_barrier
29	CAP_CODESUM	Code Summarisation	Code Review	2	0.00	organizational_readiness_barrier
30	CAP_CODESUM	Code Summarisation	Design	2	0.00	design_gap
31	CAP_CODETRANS	Code Translation	Design	2	0.00	design_gap
32	CAP_CODETRANS	Code Translation	Testing	2	0.00	organizational_readiness_barrier
33	CAP_COMMITMSG	Commit Message Generation	Coding	2	0.00	organizational_readiness_barrier

Rank	Capability	Label	SDLC	Evidence	Priority	Bucket
34	CAP_COMMITMSG	Commit Message Generation	Documentation	2	0.00	organizational_readiness_barrier
35	CAP_MULTIAGENT	Multi-Agent Collaboration	Documentation	2	0.00	adoption_lag
36	CAP_PROGREPAIR	Program Repair / Bug Fixing	Design	2	0.00	design_gap
37	CAP_PROGREPAIR	Program Repair / Bug Fixing	Requirements	2	0.00	design_gap
38	CAP_REQENG	Requirements Engineering	CI/CD	2	0.00	design_gap
39	CAP_REQENG	Requirements Engineering	Code Review	2	0.00	design_gap
40	CAP_SELFREFLECT	Self-Reflection & Iterative Refinement	Design	2	0.00	design_gap
41	CAP_SYSDSIGN	System Design & Architecture	Project Management	2	0.00	design_gap
42	CAP_VULNDET	Vulnerability Detection	Documentation	2	0.00	organizational_readiness_barrier

C.5 Research agenda — per-call detail

We define a structured research agenda comprising seven formalized Calls to Action. Each Call operates as a pre-registered proposal for a future primary study, specifying a methodological classification drawn from Wieringa’s [79] taxonomy, a target gap set (or scope extension), a research-question template, and a targeted population fit. Calls 1 through 6 are corpus-internal, explicitly operationalizing the 42 identified gaps and 12 empty cells: Calls 1–5 target the 13 highest-priority gaps (priority ≥ 1.00), while Call 6 serves as an integrative Evaluation Research initiative covering a cluster of ten moderate-priority (priority = 0.50) knowledge-management gaps. Call 7 extends the agenda beyond the internal gap matrix to address the Pro-code-dominance transferability boundary; it is framed as a downstream taxonomy construction and validation study rather than a standard gap-targeted investigation.

Call 1: AI Code Completion in Early-Lifecycle Phases

- **Methodological Class:** Evaluation Research.
- **Targeted Gaps:** Ranks 1–2 (Two priority-1.50 gaps: CAP_CODECOMP $\times$ {Design, Requirements}).
- **Research Question Template:** How do practitioners utilize code-completion tools during requirements elicitation and software design, and how do these usage patterns diverge from the coding-stage applications that currently dominate the literature?
- **Population Fit:** Professional software engineers and business analysts within industry contexts.

Call 2: AI Code Translation Beyond the Coding Boundary

- **Methodological Class:** Validation Research (at Requirements); Evaluation Research (at CI/CD and Project Management).
- **Targeted Gaps:** Ranks 3–5 (Three priority-1.50 gaps: CAP_CODETRANS $\times$ {CI/CD, Project Management, Requirements}).

- **Research Question Template:** How do legacy-modernization initiatives integrate AI code translation into migration pipelines and project management, and under what conditions does automated translation reliably preserve specification intent?
- **Population Fit:** Enterprise modernization teams within industry contexts.

Call 3: Agentic Self-Reflection Outside of Coding

- **Methodological Class:** Validation Research.
- **Targeted Gaps:** Ranks 6–8 (Three priority-1.50 gaps: CAP_SELFREFLECT $\times$ {Code Review, Documentation, Project Management}).
- **Research Question Template:** To what extent does agentic self-reflection yield measurably superior outcomes for code-review synthesis, documentation generation, and project-management task decomposition compared to single-pass baselines?
- **Population Fit & Constraints:** Designed for researcher-led lab studies or industry pilot programs. Following Kumar et al. [36], studies must pre-register a strict falsifiability constraint: self-reflection is credited only if its output-quality advantage persists under user-disagreement conditions, actively controlling for sycophancy and overconfidence failure modes.

Call 4: Multi-Agent Orchestration at Under-Studied SDLC Intersections

- **Methodological Class:** Validation Research.
- **Targeted Gaps:** Ranks 9–11 (Three priority-1.00 gaps: CAP_MULTIAGENT $\times$ {CI/CD, Code Review, Design}).
- **Research Question Template:** How effectively do multi-agent orchestration frameworks perform as CI/CD pipeline coordinators, collaborative code-review committees, and design-stage ideation partners, and which coordination patterns exhibit the greatest robustness?
- **Population Fit & Constraints:** Targets researcher-led lab studies or early-adopter industry partnerships. Studies must pre-register two falsifiable metrics from Kumar et al. [36]: the unsolicited-agent-action rate (per prompt-turn) to measure coordination quality, and the success differential between one-shot versus incremental coordination patterns across the three target SDLC intersections.

Call 5: AI Vulnerability Detection Beyond Coding

- **Methodological Class:** Evaluation Research.
- **Targeted Gaps:** Ranks 12–13 (Two priority-1.00 gaps: CAP_VULNDET $\times$ {CI/CD, Project Management}).
- **Research Question Template:** How is AI-augmented vulnerability detection embedded within CI/CD pipelines and enterprise security management, and which organizational factors best predict the effective triage of AI-generated security alerts?
- **Population Fit & Context:** Enterprise security teams within industry contexts. This Call is empirically supported by Gao et al. [23], who identify security-alert triage as a critical organizational load-bearing challenge in the LLM era.

Call 6: Knowledge Management in Non-Native Activities

- **Methodological Class:** Evaluation Research.
- **Targeted Gaps:** A cluster of ten priority-0.50 gaps spanning CAP_CODESEARCH $\times$ {Code Review, Debugging, Documentation, Testing}, CAP_CODESUM $\times$ {Requirements}, and CAP_COMMITMSG $\times$ {CI/CD, Code Review, Debugging, Project Management, Testing}.

- **Research Question Template:** Outside the primary coding flow, when and how do practitioners invoke AI-driven code search, summarization, and commit-message generation? Crucially, what marginal value or friction do these tools introduce at critical decision points in review, testing, debugging, and deployment?
- **Population Fit:** Mixed-population field studies encompassing professional developers, open-source maintainers, and business analysts.

Call 7: Taxonomy of AI-Agent Architectures Beyond Pro-Code Environments

- **Methodological Class:** Evaluation Research, preceded by a Multivocal Literature Review (MLR) [24]. The MLR is a necessary prerequisite because critical architectural evidence (vendor documentation, platform frameworks, and grey-literature practitioner reports) remains severely underrepresented in peer-reviewed venues.
- **Targeted Focus:** Addresses an unranked matrix gap motivated entirely by transferability boundary IV (§C.2: Pro-code dominance at 84.1%) and practitioner-reported scenarios logged as hypotheses in the decision register.
- **Research Question Template:** Along which architectural axis—orthogonal to the delegation-depth axis defined in §4.2—do AI-agent tools partition in practice? How do these positions map to user roles (developer, citizen developer, researcher, end-user), triggering domains (business-process vs. SE-lifecycle), and cross-paradigm integrations?
- **Hypothesis Testing:** Candidate categories (e.g., generalist AI authoring IDEs, horizontal workflow orchestrators, vertical SaaS with embedded AI, agent-construction frameworks, and deployed personal AI assistants) must be treated as falsifiable hypotheses. The final architectural taxonomy will be established empirically via the MLR and a companion practitioner survey, not assumed a priori.
- **Population Fit & Instruments:** Requires a mixed-population approach with a deliberate over-sampling of non-Pro-code demographics to explicitly counter corpus boundary IV. The execution package includes Garousi et al.'s MLR guidelines [24], a platform-documentation-analysis protocol, and a practitioner survey [33].

Empty-Cell Appendix and Interpretive Boundary

The 12 matrix intersections lacking empirical evidence are systematically partitioned into two distinct categories based on semantic plausibility. Seven cells are designated as candidate exploratory studies where the capability-by-activity pairing remains theoretically sound (e.g., CAP_CODESUM applied to Project Management for stakeholder reporting, or CAP_CODESEARCH in CI/CD for artifact discovery). Conversely, five cells are classified as architecturally inapplicable (e.g., CAP_COMMITMSG mapped to Requirements or Design, as commit messages intrinsically presuppose existing code artifacts). The authoritative classification for all 12 cells is maintained in `artifacts/analysis/research_agenda.md`, and this 7-to-5 split is formally logged in the decision register as a singular interpretive judgment. Consequently, architectural inapplicability is reported as a conclusive interpretive finding, not as a target for future research.

C.6 Decision-register schema and sample slice

The Decision Register and Audit Trail. To ensure strict methodological transparency, we maintained a decision register: an append-only audit log documenting every non-trivial classification judgment. The underlying schema systematically captures the timestamp, phase, `paper_id`, decision, `rule_applied`, rationale, and `rater_initials`. To preserve the integrity of the audit trail, records are never modified in place; instead, corrections are strictly appended as new rows (flagged with the decision revise) that explicitly reference the preceding entry.

Phase Distribution and Longitudinal Tracking. At the initial manuscript-draft freeze (April 20, 2026), the register contained 397 distinct rows distributed across five methodological phases: Phase 1 (2), Phase 2 (24), Phase 3 (325), Phase 4 (4), and Phase 5 (42). The heavy concentration in Phase 3 is predominantly driven by full-text F1 revisions (323 rows). Conversely, Phase 5 consists of single, per-gap hypothesis entries (recorded as `hypothesis_for_erp3`) that map qualitative bucket labels to the identified matrix gaps.

C.7 Data Provenance — source map for every load-bearing numeric claim

Data Provenance Registry. This section serves as the authoritative registry for all quantitative data cited throughout this ERP. Each entry systematically documents a specific claim, its exact numerical value, the source file that authoritatively generates or houses that value, and the applied verification method. To optimize readability while preserving strict traceability, inline data pointers in the main text are provided strictly upon first appearance; all subsequent references rely on this Data Provenance table as the canonical back-reference. All specified file paths are relative to the root directory of the replication repository.

Table 12: Corpus and extraction counts

Claim	Value	Source	Verification
Primary empirical studies (total corpus)	640	<code>artifacts/screening/included_set.csv</code>	Row count
Papers with full-text retrieved	258	<code>artifacts/extraction/retrieval_status.csv</code>	Filter <code>status == retrieved</code>
Papers in Mode A (full-text PDF)	208	<code>artifacts/extraction/extraction_matrix.csv</code>	<code>extraction_source == "fulltext"</code>
Papers in Mode B (abstract-only)	432	<code>artifacts/extraction/extraction_matrix.csv</code>	<code>extraction_source == "abstract_only"</code>
Papers with ≥ 1 verbatim passage (RQ2 grounding base)	290	<code>artifacts/extraction/open_codes_pass1.csv</code>	Distinct <code>paper_id</code> count
Papers with no passages (RQ1/RQ3 only)	350	Derived	$= 640 - 290$
Pass-1 open codes	1,016	<code>artifacts/extraction/open_codes_pass1.csv</code>	Row count
Canonical labels after Pass-2 clustering	707	<code>artifacts/synthesis/consolidated_codes.csv</code>	Row count
Capability annotations (total)	776	<code>artifacts/extraction/capability_annotations.csv</code>	Row count
Distinct papers carrying capability annotations	384	<code>artifacts/extraction/capability_annotations.csv</code>	Distinct <code>paper_id</code> count

Table 13: Facet distributions (F1–F5)

Claim	Value	Source	Verification
F2 Experiment	249 (38.9%)	<code>artifacts/extraction/extraction_matrix.csv</code>	<code>f2_research_methodology == Experiment;</code> denominator 640
F2 Mixed	139 (21.7%)	same	same
F2 Case Study	92 (14.4%)	same	same
F2 Interview	20 (3.1%)	same	same
F2 Field Study	14 (2.2%)	same	same
F3 Student	133	<code>artifacts/extraction/extraction_matrix.csv</code>	<code>f3_population == Student</code>
F3 Professional SWE	128	same	<code>f3_population == Professional SWE</code>

Claim	Value	Source	Verification
F3 Mixed	111	same	<code>f3_population == Mixed</code>
F3 coverage (non-N/A)	61.6%	same	<code>Count non-N/A ÷ 640 = 394 ÷ 640</code>
F5 Pro-code	538 (84.1%)	<code>artifacts/extraction/extraction_matrix.csv</code>	<code>f5_tool_paradigm == Pro-code; denominator 640</code>
F5 No-code	72 (11.3%)	same	<code>f5_tool_paradigm == No-code</code>
F5 Low-code	30 (4.7%)	same	<code>f5_tool_paradigm == Low-code</code>

Table 14: Gap matrix and priority rankings

Claim	Value	Source	Verification
Gap-matrix total cells (19 × 9)	171	<code>artifacts/analysis/rq3_gap_matrix/gap_matrix.csv</code>	Row count
Normal cells (evidence > P25)	117 (68.4%)	same	<code>Filter is_gap == False AND is_empty == False</code>
Gap cells (evidence ≤ P25, > 0)	42 (24.6%)	same	<code>Filter is_gap == True</code>
Empty cells (evidence = 0)	12 (7.0%)	same	<code>Filter is_empty == True</code>
P25 threshold (non-zero cells)	2.0	<code>artifacts/analysis/prioritised_gaps.md</code>	Header block
Maximum cell	CAP_CODEGE N × Coding = 156	<code>artifacts/analysis/rq3_gap_matrix/gap_matrix.csv</code>	<code>max(evidence_count)</code>
CI/CD activity — gaps	5 of 42	same	<code>Filter sdlc_activity == CI/CD AND is_gap == True</code>
CI/CD activity — empty	4 of 12	same	<code>Filter sdlc_activity == CI/CD AND is_empty == True</code>
Priority-1.50 gaps (ranks 1–8)	8	<code>artifacts/analysis/prioritised_gaps.md</code>	Ranked list rows 1–8
Priority-1.00 gaps (ranks 9–13)	5	same	Ranked list rows 9–13
Priority ≥ 1.00 gaps (total)	13	same	Sum of priority-1.50 + priority-1.00
Priority-0.50 gaps (ranks 14–23)	10	same	Ranked list rows 14–23
Priority-0.00 gaps (ranks 24–42, borderline at P25)	19	same	Ranked list rows 24–42
Organisational-readiness bucket	22 of 42 (52.4%)	<code>decision_register.csv</code>	Phase-5 <code>hypothesis_for_erp3</code> rationale filter
Design-gap bucket	14 of 42 (33.3%)	same	same
Adoption-lag bucket	6 of 42 (14.3%)	same	same
Gap-matrix total cells (19 × 9)	171	<code>artifacts/analysis/rq3_gap_matrix/gap_matrix.csv</code>	Row count

Table 15: Interaction-mode taxonomy counts

Claim	Value	Source	Verification
Mode 1 Inline Completion canonical labels	4 (0.6%)	artifacts/synthesis/interaction_taxonomy.md	Mode-layer tally; denominator 707
Mode 2 Conversational Prompting	69 (9.8%)	same	same
Mode 3 Visual / Declarative Composition	9 (1.3%)	same	same
Mode 4 Review & Validation	82 (11.6%)	same	same
Mode 5 Delegated Task Execution	75 (10.6%)	same	same
Residuals	468 (66.2%)	same	same
Mode-layer saturation — no new modes in final	29 of 290 papers	artifacts/synthesis/saturation_data_mode.csv	Last 10% of processing-order sequence; verified in saturation_report_mode.md
Canonical merge cosine threshold	0.35	artifacts/protocol/extraction_schema.md	Protocol parameter

Table 16: Trustworthiness, audit, and transferability metrics

Claim	Value	Source	Verification
Paraphrase-linter violations	0 of 6,295 15-word substrings	code/dod_checks.py::check_phase7_task7_1	Most recent dispatcher run
Decision-register rows (at manuscript draft)	397	decision_register.csv	Row count at freeze timestamp 2026-04-20
Decision-register rows by phase	P1 = 2; P2 = 24; P3 = 325; P4 = 4; P5 = 42	same	Group-by phase on frozen slice
Transferability boundary iv — Pro-code dominance	84.1% of F5-profiled papers	artifacts/extraction/extraction_matrix.csv	Same cell as F5 Pro-code above
Transferability boundary v — partial F3 coverage	61.6% non-N/A	same	Same cell as F3 coverage above
Transferability boundary vi — Mode 1 thin evidence	4 of 707 canonical labels	artifacts/synthesis/interaction_taxonomy.md	Same cell as Mode 1 above
Transferability boundary vii — Mode-A/B split	290 / 350	artifacts/extraction/extraction_status.csv	Same cells as Mode A / Mode B above
Paraphrase-linter violations	0 of 6,295 15-word substrings	code/dod_checks.py::check_phase7_task7_1	Most recent dispatcher run

C.8. Methodology anchor traceability

As outlined in Section 2.4, five methodological sources inform the nine foundational design decisions of this study. The following table maps each of these decisions to its corresponding design specification, implementation code, and output artifacts. To ensure clarity, each row represents a distinct methodological decision rather than an individual

source paper. In cases where a decision lacks a dedicated computational design document, such as the four analytical trustworthiness criteria, the design specification cell is marked accordingly.

Table 17: Methodology anchor traceability

Methodology Anchor	Methodology	Purpose	Design	Code	Artifacts
SMS: 5-facet classification scheme (F1–F5)	Petersen et al. 2008, 2015[52, 53]	Pre-registered facet scheme — Contribution Type (F1), Research Methodology (F2), Population and Context (F3), SDLC Activity (F4), Agent / Tool Profile (F5) — applied consistently across every corpus paper.	design/3_2_extraction.md, design/2_5_1_screening_harness.md	code/screening_harness.py, code/extraction.py	artifacts/protocol/codebook.md (F1–F5 operational definitions); artifacts/extraction/extraction_matrix.csv (640 papers × facets); manuscript/tables/wieringa_f1_distribution.md, corpus_descriptives.md
SMS: PRISMA attrition flow	Petersen et al. 2008, 2015[52, 53]	Transparent record of search → dedup → IC5 filter → title / abstract screening → snowball → final inclusion.	design/2_7_prisma_builder.md	code/prisma_builder.py	artifacts/screening/prisma_flow.md, GeminiBeautified_PrismaFlow.png; manuscript/tables/prisma_counts.md, exclusion_counts_by_criterion.md; manuscript/figures/prisma_flow.png
SMS: Visualisation conventions (§4.1)	Petersen et al. 2008, 2015[52, 53]	Petersen-standard RQ1 visualisations — year × SDLC bubble plot, F2 methodology bar chart, F5 tool-profile stacked bar, year × SDLC heatmap.	design/5_1_analysis_rq1.md	code/analysis_rq1.py	artifacts/analysis/rq1_landscape/year_sdLC_bubble.png, f2_method_bar.png, f5_tool_profile_stacked.png, sdLC_year_heatmap.png (each with a companion *.meta.json and *_imgexplain.md); mirrored in manuscript/figures/
5-step thematic synthesis → RQ2 taxonomy	Cruzes & Dybå 2011, 2015[12, 13]	Inductively derive a mode-level taxonomy from 1,016 pass-1 codes → 707 canonical labels → 5 modes, with a saturation stopping rule.	design/4_1_coding_consolidate.md, design/4_2_interaction_taxonomy.md, design/4_2_taxonomy_classify.md, design/4_2	code/coding_consolidate.py, code/taxonomy_classify.py, code/taxonomy_classify_llmassist.py, code/taxonomy_render.py, code/saturation_report.py	artifacts/synthesis/consolidated_codes.csv, taxonomy_classifications.csv, interaction_taxonomy.md, saturation_data_mode.csv, saturation_report_mode.md, saturation_curve_mode.png

Methodology Anchor	Methodology	Purpose	Design	Code	Artifacts
Extraction–synthesis separation	Cruzes & Dybå 2011, 2015[12, 13]	Keep stable descriptive coding (extraction layer) separate from derived higher-order themes (synthesis layer) so extraction can be re-run without contaminating synthesis.	_taxonomy _classify_ll massist.md, design/4_2 _residuals_ subcategori se.md, design/4_2 _taxonomy _render.md design/3_2 _extraction. md (extraction layer); design/4_* (synthesis layer)	code/extraction.py (extraction); code/coding_consolidate.py + code/taxonomy_classify.py (synthesis)	Extraction layer: artifacts/extraction/extraction_matrix.csv, open_codes_pass1.csv, capability_annotations.csv, raw_passages/*.md. Synthesis layer: artifacts/synthesis/*
F1 contribution-type vocabulary (six Wieringa classes)	Wieringa et al. 2006[79]	Classify each paper as Evaluation Research, Validation Research, Solution Proposal, Philosophical, Opinion, or Personal Experience.	design/2_5 _1_screenin g_harness. md, design/3_2 _extraction. md	code/screening_harness.py, code/extraction.py	artifacts/protocol/codebook. md (F1 operational definitions); artifacts/extraction/extraction_matrix.csv (f1_contribution_type column); manuscript/tables/wieringa_f1_distribution.md
Evaluation / Validation Research distinction — research agenda	Wieringa et al. 2006[79]	Type every research-agenda Call as Evaluation Research or Validation Research; each Call names its class in the per-Call record.	design/5_3 _prioritize_ gaps.md	code/prioritize_gaps.py	artifacts/analysis/prioritised_gaps.md, artifacts/analysis/research_agenda.md (per-Call records); manuscript/draft_v2.md §5.3; manuscript/appendix_v2.md §C.5
Backward snowballing (Phase 2)	Ali & Petersen 2014 [4]	Extract references from the five seed surveys and check against the existing corpus. Phase-2 result: 778 references extracted, 40 overlap with corpus, zero new inclusions.	design/2_6 _snowball. md	code/snowball.py	artifacts/search/raw/snowball_seeds_refs.csv

Methodology Anchor	Methodology	Purpose	Design	Code	Artifacts
Four qualitative-research criteria (credibility, confirmability, dependability, transferability)	Lincoln & Guba 1985, adapted to software engineering by Cruzes & Dybå 2011[12, 40]	Structure the trustworthiness audit in §6. Each criterion has its own audit artefact with specific operational checks.	No dedicated design document (criteria are analytical rather than computational)	code/dod_checks.py (dispatcher), code/paraphrase_linter.py (credibility), code/trace_checker.py (confirmability), code/saturation_report.py (dependability)	artifacts/trustworthiness/credibility_statement.md, confirmability_annex.md, dependability_audit.md, transferability.md, threshold_sensitivity.md

APPENDIX D: RQ3 AND TRUSTWORTHINESS ARTEFACTS

D.1 SDLC Activity × Tool Modality in the Ranked Full-Text Subset

Cross-tabulation of full-text-extracted papers from SCIS-ranked venues, indexed by SDLC activity (F4, rows) and tool modality (F5, columns).

Table 18: SDLC Activity × Tool Modality in the Ranked Full-Text Subset

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
Requirements	<i>n</i> = 2 [23, 28] Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i> Jensen et al. 2025 (10.1007/s10257-025-00704-7). <i>"Managing expectations towards AI tools for software development: a multiple-case study"</i>	<i>n</i> = 7 [7, 11, 19, 23, 28, 38, 67] Awasthi et al. 2026 (10.1145/3772318.3790459). <i>"TurnStyle: A Framework for Analyzing Human Conversational Behaviors to Predict Success in LLM-Assisted Tasks"</i> Cheng et al. 2024 (10.1145/3638247). <i>"Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"</i> Flamino et al. 2025 (10.1038/s41598-025-98378-1). <i>"Testing the limits of large language models in debating humans"</i> Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i>	<i>n</i> = 3 [11, 23, 28] Cheng et al. 2024 (10.1145/3638247). <i>"Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"</i> Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i> Jensen et al. 2025 (10.1007/s10257-025-00704-7). <i>"Managing expectations towards AI tools for software development: a multiple-case study"</i>	<i>n</i> = 1 [19] Flamino et al. 2025 (10.1038/s41598-025-98378-1). <i>"Testing the limits of large language models in debating humans"</i>

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
Design		Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"		
		Kuo et al. 2026 (10.1145/3772318.3790500). "Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations"		
		Truss 2026 (10.1145/3772363.3798543). "PersonaCite: VoC-Grounded Interviewable Agentic Synthetic AI Personas for Verifiable User and Design Research"		
	$n = 4$ [8, 23, 28, 70]	$n = 8$ [8, 11, 23, 28, 38, 43, 67, 70]	$n = 4$ [11, 23, 28, 70]	—
	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	
	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	
	Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"	
		Kuo et al. 2026 (10.1145/3772318.3790500). "Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations"		
		Liu et al. 2024 (10.1145/3613905.3650756). "We Need Structured Output: Towards User-centered Constraints on Large Language Model Output"		
		Truss 2026 (10.1145/3772363.3798543). "PersonaCite: VoC-Grounded Interviewable Agentic Synthetic AI Personas for Verifiable User and Design Research"		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
Coding		Vigh et al. 2026 (10.1145/3772363.3799079). " <i>"AI does not understand the real world."</i> : <i>AI Augmented Software Development</i> "		
	<i>n</i> = 17 [3, 8, 14, 18, 20, 23, 27, 28, 35, 50, 63, 69, 70, 76, 81, 85, 86]	<i>n</i> = 38 [3, 5–8, 10, 11, 15, 17, 20, 23, 25, 28–30, 38, 43–45, 47, 49, 50, 54–56, 59–61, 63–65, 68, 70, 72, 74, 76, 82, 84]	<i>n</i> = 16 [3, 11, 14, 18, 20, 23, 27, 28, 35, 50, 63, 69, 70, 76, 85, 86]	<i>n</i> = 5 [29, 65, 74, 81, 84]
	Ali et al. 2025 (10.1007/s40692-025-00375-w). " <i>Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model</i> "	AlOmar et al. 2024 (10.1145/3643991.3645081). " <i>How to Refactor this Code? An Exploratory Study on Developer-ChatGPT Refactoring Conversations</i> "	Ali et al. 2025 (10.1007/s40692-025-00375-w). " <i>Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model</i> "	Jiang et al. 2025 (10.14778/3750601.3750610). " <i>SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence</i> "
	Banh et al. 2025 (10.1016/j.infsof.2025.107751). " <i>Copiloting the future: How generative AI transforms Software Engineering</i> "	Ali et al. 2025 (10.1007/s40692-025-00375-w). " <i>Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model</i> "	Cheng et al. 2024 (10.1145/3638247). " <i>Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains</i> "	Torun et al. 2026 (10.1145/3806655). " <i>Past, Present, and Future of Bug Tracking in the Generative AI Era</i> "
	Dakhel et al. 2023 (10.1016/j.jss.2023.111734). " <i>GitHub Copilot AI pair programmer: Asset or Liability?</i> "	Almeida et al. 2024 (10.1145/3674805.3690746). " <i>Automatic Library Migration Using Large Language Models: First Results</i> "	Dakhel et al. 2023 (10.1016/j.jss.2023.111734). " <i>GitHub Copilot AI pair programmer: Asset or Liability?</i> "	Watanabe et al. 2026 (10.1145/3798166). " <i>On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub</i> "
	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). " <i>LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation</i> "	Awasthi et al. 2026 (10.1145/3772318.3790459). " <i>TurnStyle: A Framework for Analyzing Human Conversational Behaviors to Predict Success in LLM-Assisted Tasks</i> "	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). " <i>LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation</i> "	Xue et al. 2026 (10.1145/3793252). " <i>Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset</i> "
	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). " <i>The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired</i> "	Banh et al. 2025 (10.1016/j.infsof.2025.107751). " <i>Copiloting the future: How generative AI transforms Software Engineering</i> "	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). " <i>The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired</i> "	Zhang et al. 2024 (10.1109/TSE.2024.3364675). " <i>Automatic Commit Message Generation: A Critical Review and Directions for Future Work</i> "
	Gao et al. 2025 (10.1145/3712005). " <i>The Current Challenges of Software Engineering in the Era of Large Language Models</i> "	Cheng et al. 2025 (10.1007/s10515-025-00492-x). " <i>A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair</i> "	Gao et al. 2025 (10.1145/3712005). " <i>The Current Challenges of Software Engineering in the Era of Large Language Models</i> "	
	Huang et al. 2022 (10.1109/TKDE.2024.3436883). " <i>SE Factual Knowledge in Frozen Giant Code Model: A Study on FQN and its Retrieval</i> "	Cao et al. 2025 (10.1007/s10515-025-00492-x). " <i>A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair</i> "	Huang et al. 2022 (10.1109/TKDE.2024.3436883). " <i>SE Factual Knowledge in Frozen Giant Code Model: A Study on FQN and its Retrieval</i> "	
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). " <i>Managing expectations towards AI tools for software development: a multiple-case study</i> "	Cheng et al. 2024 (10.1145/3638247). " <i>Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains</i> "	Jensen et al. 2025 (10.1007/s10257-025-00704-7). " <i>Managing expectations towards AI tools for software development: a multiple-case study</i> "	
	Koyanagi et al. 2024 (10.1145/3643991.3644917). " <i>Exploring the</i>	Elinzano et al. 2024 (10.1145/3704611.3704621). " <i>Towards an AI-Assisted Software Developer: The Perspective</i>	Koyanagi et al. 2024 (10.1145/3643991.3644917). " <i>Exploring the Effect of Multiple Natural Languages on Code Suggestion Using GitHub Copilot</i> "	

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	<i>Effect of Multiple Natural Languages on Code Suggestion Using GitHub Copilot"</i>	<i>on Use of ChatGPT for Programming in the Philippines"</i>	Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i>	
	Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i>	Esnaola et al. 2026 (10.1007/s10664-026-10813-7). <i>"Can generative AI bridge the gap? A quasi-experimental study of non-programmers with AI vs. programmers without AI"</i>	Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i>	
	Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i>	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). <i>"The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"</i>	Vaithilingam et al. 2022 (10.1145/3491101.3519665). <i>"Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models"</i>	
	Vaithilingam et al. 2022 (10.1145/3491101.3519665). <i>"Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models"</i>	Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i>	Vigh et al. 2026 (10.1145/3772363.3799079). <i>"AI does not understand the real world.": AI Augmented Software Development"</i>	
	Vigh et al. 2026 (10.1145/3772363.3799079). <i>"AI does not understand the real world.": AI Augmented Software Development"</i>	Hannig et al. 2026 (10.1145/3772318.3790622). <i>"Campus AI vs. Commercial AI: Comparing How Students and Employees Perceive their University's LLM Chatbot vs. ChatGPT"</i>	Weisz et al. 2025 (10.1145/3706599.3706670). <i>"Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"</i>	
	Weisz et al. 2025 (10.1145/3706599.3706670). <i>"Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"</i>	Jensen et al. 2025 (10.1007/s10257-025-00704-7). <i>"Managing expectations towards AI tools for software development: a multiple-case study"</i>	Zhou et al. 2024 (10.1016/j.jss.2024.112204). <i>"Exploring the Problems, their Causes and Solutions of AI Pair Programming: A Study on GitHub and Stack Overflow"</i>	
	Xue et al. 2026 (10.1145/3793252). <i>"Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset"</i>	Jiang et al. 2025 (10.14778/3750601.3750610). <i>"SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence"</i>	Zhuo et al. 2025 (10.1109/tse.2026.3651566). <i>"Identifying and Mitigating API Misuse in Large Language Models"</i>	
	Zhou et al. 2024 (10.1016/j.jss.2024.112204). <i>"Exploring the Problems, their Causes and Solutions of AI Pair Programming: A Study on GitHub and Stack Overflow"</i>	Jin et al. 2024 (10.1145/3643991.3645074). <i>"Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation"</i>		
	Zhuo et al. 2025 (10.1109/tse.2026.3651566). <i>"Identifying and Mitigating API Misuse in Large Language Models"</i>	Kuo et al. 2026 (10.1145/3772318.3790500). <i>"Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations"</i>		
		Liu et al. 2024 (10.1109/TSE.2024.3392499). <i>"No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT"</i>		
		Liu et al. 2024 (10.1145/3613905.3650756). <i>"We Need Structured Output: Towards User-</i>		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		centered Constraints on Large Language Model Output"		
		Monteiro et al. 2025 (10.1002/spe.3432). <i>"NoCodeGPT: A No-Code Interface for Building Web Apps with Language Models"</i> Nofal et al. 2025 (10.1016/j.caeai.2024.100347). <i>"AI-enhanced interview simulation in the metaverse: Transforming professional skills training through VR and generative conversational AI"</i> Ouyang et al. 2025 (10.1145/3697010). <i>"An Empirical Study of the Non-Determinism of ChatGPT in Code Generation"</i> Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i> Popovici 2024 (10.1080/10447318.2023.2269006). <i>"ChatGPT in the classroom. Exploring its potential and limitations in a Functional Programming course."</i> Przymus et al. 2024 (10.1145/3643991.3645073). <i>"How I Learned to Stop Worrying and Love ChatGPT"</i> Raj et al. 2024 (10.1145/3643991.3645075). <i>"The role of library versions in Developer-ChatGPT conversations"</i> Sajadi et al. 2025 (10.1007/s10664-025-10658-6). <i>"Do LLMs consider security? an empirical study on responses to programming questions"</i> Serafini et al. 2025 (10.1145/3706598.3713989). <i>"Exploring the Impact of Intervention Methods on Developers' Security Behavior in a Manipulated ChatGPT Study"</i> Silva et al. 2025 (10.1016/j.jss.2025.112541). <i>"LLMs and Stack Overflow Discussions: Reliability, Impact, and Challenges"</i>		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i>		
		Tie et al. 2026 (10.1145/3801972). <i>"Should I Give Up Now?" Investigating LLM Pitfalls in Software Engineering"</i>		
		Torun et al. 2026 (10.1145/3806655). <i>"Past, Present, and Future of Bug Tracking in the Generative AI Era"</i>		
		Tufano et al. 2024 (10.1145/3643991.3644918). <i>"Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study"</i>		
		Vigh et al. 2026 (10.1145/3772363.3799079). <i>"AI does not understand the real world.": AI Augmented Software Development"</i>		
		Wang et al. 2025 (10.1145/3706598.3713852). <i>"Exploring Personalized Health Support through Data-Driven, Theory-Guided LLMs: A Case Study in Sleep Health"</i>		
		Watanabe et al. 2026 (10.1145/3798166). <i>"On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"</i>		
		Weisz et al. 2025 (10.1145/3706599.3706670). <i>"Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"</i>		
		Yu et al. 2024 (10.1109/TSE.2024.3492204). <i>"Fight Fire with Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?"</i>		
		Zhang et al. 2024 (10.1109/TSE.2024.3364675). <i>"Automatic Commit Message Generation: A Critical Review and Directions for Future Work"</i>		
Testing	$n = 7$ [8, 18, 20, 23, 28, 50, 76] Banh et al. 2025 (10.1016/j.infsof.2025.107751). <i>"Copiloting"</i>	$n = 15$ [5, 8, 11, 20, 23, 28, 38, 50, 54, 57, 65, 68, 74, 76, 82] Almeida et al. 2024 (10.1145/3674805.3690746). <i>"Automatic"</i>	$n = 7$ [11, 18, 20, 23, 28, 50, 76]	$n = 6$ [31, 46, 58, 65, 74, 83] Ju et al. 2024 (10.1109/APSEC65559.2024.00017). <i>"A Study"</i>

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	<i>the future: How generative AI transforms Software Engineering"</i>	<i>Library Migration Using Large Language Models: First Results"</i>	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	<i>of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps"</i>
	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). "LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation"	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). "LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation"	Nass et al. 2023 (10.1002/stvr.1893). "Improving web element localization by using a large language model"
	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Roitero et al. 2025 (10.1145/3726302.3729960). "Efficiency and Effectiveness of LLM-Based Summarization of Evidence in Crowdsourced Fact-Checking"
	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era"
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"
	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Zhang et al. 2023 (10.1145/3706598.3713600). "AppAgent: Multimodal Agents as Smartphone Users"
	Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"	Kuo et al. 2026 (10.1145/3772318.3790500). "Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations"	Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"	
		Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"		
		Popovici 2024 (10.1080/10447318.2023.2269006). "ChatGPT in the classroom. Exploring its potential and limitations in a Functional Programming course."		
		Rastogi et al. 2023 (10.1145/3600211.3604712). "Supporting Human-AI Collaboration in Auditing LLMs with LLMs"		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study" Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub" Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise" Yu et al. 2024 (10.1109/TSE.2024.3492204). "Fight Fire with Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?"		
Code Review	<i>n</i> = 5 [8, 23, 50, 70, 81]	<i>n</i> = 11 [2, 8, 22, 23, 25, 50, 54, 55, 68, 70, 78]	<i>n</i> = 3 [23, 50, 70]	<i>n</i> = 4 [32, 41, 77, 81]
	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering" Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development" Xue et al. 2026 (10.1145/3793252). "Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset"	Alami et al. 2025 (10.1145/3721127). "Accountability in Code Review: The Role of Intrinsic Drivers and the Impact of LLMs" Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering" Fu et al. 2023 (10.1109/apsec60848.2023.00085). "ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?" Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Hannig et al. 2026 (10.1145/3772318.3790622). "Campus AI vs. Commercial AI: Comparing How Students and Employees Perceive their University's LLM Chatbot vs. ChatGPT"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"	Kang et al. 2025 (10.1007/s10664-024-10594-x). "Explainable automated debugging via large language model-driven scientific debugging" Liu et al. 2025 (10.1109/MSR66628.2025.00043). "Too Noisy To Learn: Enhancing Data Quality for Code Review Comment Generation" Wen et al. 2024 (10.1145/3653718). "Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We?" Xue et al. 2026 (10.1145/3793252). "Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset"

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Popovici 2024 (10.1080/10447318.2023.2269006). "ChatGPT in the classroom. Exploring its potential and limitations in a Functional Programming course." Przymus et al. 2024 (10.1145/3643991.3645073). "How I Learned to Stop Worrying and Love ChatGPT" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT[™]s Usage in Open Source Projects: A Mining-based Study" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development" Widyasari et al. 2024 (10.1145/3708518). "Explaining Explanations: An Empirical Study of Explanations in Code Reviews"		
Debugging	$n = 6$ [3, 8, 20, 50, 63, 86] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model" Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering" Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants"	$n = 24$ [3, 7–11, 15, 16, 20, 22, 30, 44, 50, 56, 57, 59, 61, 63–65, 68, 74, 80, 82] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model" Awasthi et al. 2026 (10.1145/3772318.3790459). "TurnStyle: A Framework for Analyzing Human Conversational Behaviors to Predict Success in LLM-Assisted Tasks" Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	$n = 6$ [3, 11, 20, 50, 63, 86] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model" Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains" Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	$n = 7$ [16, 31, 32, 39, 65, 74, 77] Epperson et al. 2025 (10.1145/3706598.3713581). "Interactive Debugging and Steering of Multi-Agent AI Systems" Ju et al. 2024 (10.1109/APSEC65559.2024.00017). "A Study of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps" Kang et al. 2025 (10.1007/s10664-024-10594-x). "Explainable automated debugging via large language model-driven scientific debugging" Le-Cong et al. 2023 (10.1145/3716167). "Towards Reliable Evaluation of Neural Program Repair with Natural Robustness Testing"

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	<i>among South Korean Industry Developers: A survey case study with large language models</i>	Bo et al. 2025 (10.1145/3772318.3791365). <i>"Invisible Saboteurs: Sycophantic LLMs Mislead Novices in Problem-Solving Tasks"</i>	Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i>	Torun et al. 2026 (10.1145/3806655). <i>"Past, Present, and Future of Bug Tracking in the Generative AI Era"</i>
	Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i>	Cao et al. 2025 (10.1007/s10515-025-00492-x). <i>"A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair"</i>	Zhuo et al. 2025 (10.1109/tse.2026.3651566). <i>"Identifying and Mitigating API Misuse in Large Language Models"</i>	Watanabe et al. 2026 (10.1145/3798166). <i>"On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"</i>
	Zhuo et al. 2025 (10.1109/tse.2026.3651566). <i>"Identifying and Mitigating API Misuse in Large Language Models"</i>	Cheng et al. 2024 (10.1145/3638247). <i>"Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"</i>		Wen et al. 2024 (10.1145/3653718). <i>"Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We?"</i>
		Elinzano et al. 2024 (10.1145/3704611.3704621). <i>"Towards an AI-Assisted Software Developer: The Perspective on Use of ChatGPT for Programming in the Philippines"</i>		
		Epperson et al. 2025 (10.1145/3706598.3713581). <i>"Interactive Debugging and Steering of Multi-Agent AI Systems"</i>		
		Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). <i>"The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"</i>		
		Fu et al. 2023 (10.1109/apsec60848.2023.00085). <i>"ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?"</i>		
		Jin et al. 2024 (10.1145/3643991.3645074). <i>"Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation"</i>		
		Liu et al. 2024 (10.1109/TSE.2024.3392499). <i>"No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT"</i>		
		Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i>		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		Raj et al. 2024 (10.1145/3643991.3645075). <i>"The role of library versions in Developer-ChatGPT conversations"</i> Rastogi et al. 2023 (10.1145/3600211.3604712). <i>"Supporting Human-AI Collaboration in Auditing LLMs with LLMs"</i> Sajadi et al. 2025 (10.1007/s10664-025-10658-6). <i>"Do LLMs consider security? an empirical study on responses to programming questions"</i> Silva et al. 2025 (10.1016/j.jss.2025.112541). <i>"LLMs and Stack Overflow Discussions: Reliability, Impact, and Challenges"</i> Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i> Tie et al. 2026 (10.1145/3801972). <i>"Should I Give Up Now?" Investigating LLM Pitfalls in Software Engineering"</i> Torun et al. 2026 (10.1145/3806655). <i>"Past, Present, and Future of Bug Tracking in the Generative AI Era"</i> Tufano et al. 2024 (10.1145/3643991.3644918). <i>"Unveiling ChatGPT[™]s Usage in Open Source Projects: A Mining-based Study"</i> Watanabe et al. 2026 (10.1145/3798166). <i>"On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"</i> Wu et al. 2022 (10.1145/3491102.3517582). <i>"AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts"</i> Yu et al. 2024 (10.1109/TSE.2024.3492204). <i>"Fight Fire with Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?"</i>		
Documentation	$n = 4$ [28, 50, 70, 76]	$n = 9$ [15, 28, 30, 50, 68, 70, 75, 76, 84]	$n = 5$ [26, 28, 50, 70, 76]	$n = 2$ [75, 84]

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development" Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"	Elinzano et al. 2024 (10.1145/3704611.3704621). "Towards an AI-Assisted Software Developer: The Perspective on Use of ChatGPT for Programming in the Philippines" Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study" Jin et al. 2024 (10.1145/3643991.3645074). "Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development" Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub" Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise" Zhang et al. 2024 (10.1109/TSE.2024.3364675). "Automatic Commit Message Generation: A Critical Review and Directions for Future Work"	Huang et al. 2024 (10.1145/3706599.3720193). "Inspo: Writing with Crowds Alongside AI" Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development" Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"	Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub" Zhang et al. 2024 (10.1109/TSE.2024.3364675). "Automatic Commit Message Generation: A Critical Review and Directions for Future Work"
Project	$n = 1$ [28]	$n = 5$ [28, 51, 62, 65, 68]	$n = 1$ [28]	$n = 1$ [65]
Management	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools	Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era"

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	<i>for software development: a multiple-case study"</i>	<i>for software development: a multiple-case study"</i>	<i>for software development: a multiple-case study"</i>	
		Pataranutaporn et al. 2026 (10.1145/3772318.3791851). "Personal Validation Effect in LLMs: Positive AI Responses Bias Perceptions of Validity, Reliability, Personalization, and Usefulness of Fictitious Predictions" Swain et al. 2025 (10.1145/3706598.3713705). "AI on My Shoulder: Supporting Emotional Labor in Front-Office Roles with an LLM-based Empathetic Coworker" Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study"		
CI/CD	—	<i>n</i> = 3 [65, 68, 75]	—	<i>n</i> = 2 [65, 75]
		Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study" Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"		Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"